```
"""JobsComing backend application package."""

"""API 路由包。"""

"""平台帳號加密庫（FR-1d / 主程序同步 GAP-2）。

- 寫入：為用戶生成/取回 DEK → 信封加密 → 落庫（密文）
- 讀取：僅返回掩碼後的帳號標識，永不下發密文/明文
- 解密密文只發生在採集/投遞 Worker（P3/P4），通過 get_dek_for_user 取用
"""
from __future__ import annotations

from typing import Annotated

from fastapi import APIRouter, Depends, HTTPException, Request
from sqlalchemy import text
from sqlalchemy.orm import Session

from app.core.audit import log_audit
from app.core.auth import get_current_user
from app.core.security import (
    decrypt_field,
    encrypt_field,
    encrypt_json,
    get_dek_for_user,
)
from app.db import get_db
from app.schemas.accounts import PLATFORMS, AccountCreate, AccountUpdate

router = APIRouter(prefix="/accounts", tags=["accounts"])

CurrentUser = Annotated[dict, Depends(get_current_user)]
DB = Annotated[Session, Depends(get_db)]


def _mask(identifier: str) -> str:
    if len(identifier) <= 4:
        return "****"
    return identifier[:2] + "****" + identifier[-2:]


@router.get("")
def list_accounts(user: CurrentUser, db: DB) -> list[dict]:
    rows = db.execute(
        text(
            "SELECT id, platform, label, login_identifier_enc, status, last_login_at, last_error
"
            "FROM platform_accounts WHERE user_id = :uid ORDER BY created_at"
        ),
        {"uid": user["id"]},
    ).mappings().all()
    dek = get_dek_for_user(db, user["id"])
    result = []
    for r in rows:
        result.append(
            {
                "id": str(r["id"]),
                "platform": r["platform"],
                "label": r["label"],
                "login_masked": _mask(decrypt_field(dek, r["login_identifier_enc"])),
                "status": r["status"],
                "last_login_at": r["last_login_at"].isoformat() if r["last_login_at"] else None,
                "last_error": r["last_error"],
            }
        )
```

```
    return result


@router.post("", status_code=201)
def create_account(user: CurrentUser, db: DB, payload: AccountCreate, request: Request) -> dict:
    if payload.platform not in PLATFORMS:
        raise HTTPException(422, f"platform 必須是 {PLATFORMS} 之一")
    dek = get_dek_for_user(db, user["id"])
    row_id = db.execute(
        text(
            """
            INSERT INTO platform_accounts
              (user_id, platform, label, login_identifier_enc, password_enc, meta_enc)
            VALUES (:uid, :platform, :label, :ident, :pwd, :meta)
            RETURNING id
            """
        ),
        {
            "uid": user["id"],
            "platform": payload.platform,
            "label": payload.label,
            "ident": encrypt_field(dek, payload.login_identifier),
            "pwd": encrypt_field(dek, payload.password),
            "meta": encrypt_json(dek, payload.meta) if payload.meta is not None else None,
        },
    ).mappings().one()
    db.commit()
    log_audit(db, user["id"], "account.create", "platform_accounts", str(row_id["id"]),
              {"platform": payload.platform}, request.client.host if request.client else None)
    return {"id": str(row_id["id"])}


@router.patch("/{account_id}")
def update_account(
    user: CurrentUser, db: DB, account_id: str, payload: AccountUpdate, request: Request
) -> dict:
    exists = db.execute(
        text("SELECT 1 FROM platform_accounts WHERE id = :aid AND user_id = :uid"),
        {"aid": account_id, "uid": user["id"]},
    ).first()
    if not exists:
        raise HTTPException(404, "帳號不存在")
    dek = get_dek_for_user(db, user["id"])
    fields = payload.model_dump(exclude_none=True)
    sets, params = [], {"aid": account_id, "uid": user["id"]}
    if "label" in fields:
        sets.append("label = :label")
        params["label"] = fields["label"]
    if "login_identifier" in fields:
        sets.append("login_identifier_enc = :ident")
        params["ident"] = encrypt_field(dek, fields["login_identifier"])
    if "password" in fields:
        sets.append("password_enc = :pwd")
        params["pwd"] = encrypt_field(dek, fields["password"])
    if "meta" in fields:
        sets.append("meta_enc = :meta")
        params["meta"] = encrypt_json(dek, fields["meta"]) if fields["meta"] is not None else
None
    if sets:
        sets.append("status = 'pending_verify'")  # 憑據變更後需重新驗證登錄態
        db.execute(
            text(
                f"UPDATE platform_accounts SET {', '.join(sets)}, updated_at = now() "
                "WHERE id = :aid AND user_id = :uid"
            ),
```

```
            params,
        )
        db.commit()
        log_audit(db, user["id"], "account.update", "platform_accounts", account_id,
                  {}, request.client.host if request.client else None)
    return {"ok": True}


@router.delete("/{account_id}", status_code=204)
def delete_account(user: CurrentUser, db: DB, account_id: str, request: Request) -> None:
    row = db.execute(
        text("DELETE FROM platform_accounts WHERE id = :aid AND user_id = :uid RETURNING id"),
        {"aid": account_id, "uid": user["id"]},
    ).first()
    if not row:
        raise HTTPException(404, "帳號不存在")
    db.commit()
    log_audit(db, user["id"], "account.delete", "platform_accounts", account_id,
              {}, request.client.host if request.client else None)

"""管理後台端點（GAP-10）：用戶審計、適配器健康、投遞成功率、LLM 用量、審計日誌。"""
from __future__ import annotations

from typing import Annotated

from fastapi import APIRouter, Depends, HTTPException
from sqlalchemy import text
from sqlalchemy.orm import Session

from app.core.auth import get_current_user
from app.core.llm import usage as llm_usage
from app.db import get_db

router = APIRouter(prefix="/admin", tags=["admin"])

CurrentUser = Annotated[dict, Depends(get_current_user)]
DB = Annotated[Session, Depends(get_db)]


def require_admin(user: CurrentUser) -> dict:
    if user.get("role") != "admin":
        raise HTTPException(403, "需要管理員權限")
    return user


Admin = Annotated[dict, Depends(require_admin)]


@router.get("/stats")
def stats(admin: Admin, db: DB) -> dict:
    one = lambda sql, **kw: db.execute(text(sql), kw).mappings().one()  # noqa: E731
    users = one("SELECT COUNT(*) AS n FROM profiles WHERE deleted_at IS NULL")["n"]
    active_postings = one("SELECT COUNT(*) AS n FROM job_postings WHERE is_active=true")["n"]
    matches_24h = one(
        "SELECT COUNT(*) AS n FROM job_matches WHERE created_at >= now() - interval '24 hours'"
    )["n"]
    apps = one(
        """
        SELECT COUNT(*) FILTER (WHERE submitted_at >= now() - interval '24 hours') AS
submitted_24h,
               COUNT(*) AS total FROM applications
        """
    )
    open_tasks = one("SELECT COUNT(*) AS n FROM manual_tasks WHERE status='open'")["n"]
    accounts_issues = one(
```

```
        "SELECT COUNT(*) AS n FROM platform_accounts WHERE status IN
('expired','error','locked')"
    )["n"]
    return {
        "users": users,
        "active_postings": active_postings,
        "matches_24h": matches_24h,
        "applications_total": apps["total"],
        "applications_submitted_24h": apps["submitted_24h"],
        "open_manual_tasks": open_tasks,
        "platform_accounts_issue": accounts_issues,
        "llm_usage": llm_usage(),
        "collected_today": one(
            "SELECT COUNT(*) AS n FROM job_postings WHERE first_seen_at::date = now()::date"
        )["n"],
    }


@router.get("/users")
def list_users(admin: Admin, db: DB) -> list[dict]:
    rows = db.execute(
        text(
            """
            SELECT p.id, p.email, p.phone, p.role, p.created_at, p.deleted_at,
                   (SELECT COUNT(*) FROM applications a WHERE a.user_id=p.id) AS apps,
                   (SELECT COUNT(*) FROM job_matches m WHERE m.user_id=p.id) AS matches
            FROM profiles p ORDER BY p.created_at DESC LIMIT 500
            """
        )
    ).mappings().all()
    return [
        {
            "id": str(r["id"]), "email": r["email"], "phone": r["phone"], "role": r["role"],
            "created_at": r["created_at"].isoformat(),
            "disabled": r["deleted_at"] is not None,
            "applications": r["apps"], "matches": r["matches"],
        }
        for r in rows
    ]


@router.patch("/users/{user_id}")
def set_user(admin: Admin, db: DB, user_id: str, disabled: bool | None = None,
             role: str | None = None) -> dict:
    sets, params = [], {"id": user_id}
    if disabled is not None:
        sets.append("deleted_at = " + ("now()" if disabled else "NULL"))
    if role in ("user", "admin"):
        sets.append("role = :role")
        params["role"] = role
    if not sets:
        raise HTTPException(422, "無有效變更")
    row = db.execute(
        text(f"UPDATE profiles SET {', '.join(sets)} WHERE id=:id RETURNING id"),
        params,
    ).first()
    if not row:
        raise HTTPException(404, "用戶不存在")
    db.commit()
    return {"ok": True}


@router.get("/adapters")
def adapters(admin: Admin, db: DB) -> list[dict]:
    rows = db.execute(
```

```
        text("SELECT * FROM adapter_health ORDER BY adapter")
    ).mappings().all()
    return [
        {
            "adapter": r["adapter"], "last_run_at": r["last_run_at"].isoformat() if
r["last_run_at"] else None,
            "last_success_at": r["last_success_at"].isoformat() if r["last_success_at"] else
None,
            "success_count": r["success_count"], "fail_count": r["fail_count"],
            "last_error": r["last_error"], "avg_duration_ms": r["avg_duration_ms"],
        }
        for r in rows
    ]


@router.get("/audit")
def audit(admin: Admin, db: DB, limit: int = 100) -> list[dict]:
    rows = db.execute(
        text(
            """
            SELECT a.created_at, a.action, a.entity, a.entity_id, a.meta, p.email
            FROM audit_logs a LEFT JOIN profiles p ON p.id = a.actor_user_id
            ORDER BY a.created_at DESC LIMIT :lim
            """
        ),
        {"lim": min(limit, 500)},
    ).mappings().all()
    return [
        {
            "at": r["created_at"].isoformat(), "actor": r["email"], "action": r["action"],
            "entity": r["entity"], "entity_id": r["entity_id"], "meta": r["meta"],
        }
        for r in rows
    ]


@router.get("/collect-jobs")
def collect_jobs(admin: Admin, db: DB, limit: int = 50) -> list[dict]:
    rows = db.execute(
        text(
            "SELECT kind, status, payload, scheduled_at, started_at, finished_at, error "
            "FROM collect_jobs ORDER BY created_at DESC LIMIT :lim"
        ),
        {"lim": min(limit, 200)},
    ).mappings().all()
    return [
        {
            "kind": r["kind"], "status": r["status"], "payload": r["payload"],
            "started_at": r["started_at"].isoformat() if r["started_at"] else None,
            "finished_at": r["finished_at"].isoformat() if r["finished_at"] else None,
            "error": r["error"],
        }
        for r in rows
    ]
```

```
"""投遞記錄與觸發端點（FR-3b/3c）。"""
from __future__ import annotations

from typing import Annotated

from fastapi import APIRouter, Depends, HTTPException
from sqlalchemy import text
from sqlalchemy.orm import Session

from app.core.auth import get_current_user
```

```
from app.db import get_db
from app.services.apply_engine import (
    create_applications_from_matches,
    process_queue,
    run_application,
)

router = APIRouter(prefix="/applications", tags=["applications"])

CurrentUser = Annotated[dict, Depends(get_current_user)]
DB = Annotated[Session, Depends(get_db)]


@router.get("")
def list_applications(
    user: CurrentUser, db: DB, status: str | None = None,
    limit: int = 20, offset: int = 0,
) -> dict:
    limit = min(max(limit, 1), 100)
    rows = db.execute(
        text(
            """
            SELECT a.id, a.status, a.status_reason, a.attempts_count, a.next_retry_at,
                   a.submitted_at, a.external_ref, a.created_at,
                   p.company_name, p.title, p.location, p.apply_url, p.source,
                   m.match_score, rv.id AS resume_version_id
            FROM applications a
            JOIN job_postings p ON p.id = a.posting_id
            LEFT JOIN job_matches m ON m.id = a.match_id
            LEFT JOIN resume_versions rv ON rv.id = a.resume_version_id
            WHERE a.user_id = :uid AND (CAST(:st AS text) IS NULL OR CAST(a.status AS text)
= :st)
            ORDER BY a.created_at DESC
            LIMIT :lim OFFSET :off
            """
        ),
        {"uid": user["id"], "st": status, "lim": limit, "off": offset},
    ).mappings().all()
    total = db.execute(
        text(
            """
            SELECT COUNT(*) AS n FROM applications a
            WHERE a.user_id = :uid AND (CAST(:st AS text) IS NULL OR CAST(a.status AS text)
= :st)
            """
        ),
        {"uid": user["id"], "st": status},
    ).mappings().one()["n"]
    return {
        "items": [
            {
                "id": str(r["id"]),
                "status": r["status"],
                "status_reason": r["status_reason"],
                "attempts_count": r["attempts_count"],
                "next_retry_at": r["next_retry_at"].isoformat() if r["next_retry_at"] else None,
                "submitted_at": r["submitted_at"].isoformat() if r["submitted_at"] else None,
                "external_ref": r["external_ref"],
                "created_at": r["created_at"].isoformat(),
                "company_name": r["company_name"],
                "title": r["title"],
                "location": r["location"],
                "apply_url": r["apply_url"],
                "source": r["source"],
                "match_score": r["match_score"],
```

```
                "resume_version_id": str(r["resume_version_id"]) if r["resume_version_id"] else
None,
            }
            for r in rows
        ],
        "total": total,
        "limit": limit,
        "offset": offset,
    }


@router.post("/run", status_code=202)
def run_now(user: CurrentUser, db: DB) -> dict:
    """手動觸發：按當前設置從匹配結果生成投遞隊列並立即處理。"""
    created = create_applications_from_matches(db, user["id"])
    processed = process_queue(db, user_id=user["id"])
    return {"created": created, "processed": processed}


@router.post("/{application_id}/retry", status_code=202)
def retry_application(user: CurrentUser, db: DB, application_id: str) -> dict:
    row = db.execute(
        text("SELECT status FROM applications WHERE id=:id AND user_id=:uid"),
        {"id": application_id, "uid": user["id"]},
    ).mappings().first()
    if not row:
        raise HTTPException(404, "投遞記錄不存在")
    if row["status"] in ("submitted", "in_progress"):
        return {"skipped": row["status"]}
    out = run_application(db, application_id, force=True)
    return out


@router.post("/{application_id}/abort", status_code=200)
def abort_application(user: CurrentUser, db: DB, application_id: str) -> dict:
    row = db.execute(
        text(
            "UPDATE applications SET status='aborted', status_reason='用戶手動取消',
updated_at=now() "
            "WHERE id=:id AND user_id=:uid AND status NOT IN ('submitted','aborted') RETURNING
id"
        ),
        {"id": application_id, "uid": user["id"]},
    ).first()
    if not row:
        raise HTTPException(404, "投遞記錄不存在或已結束")
    db.commit()
    return {"ok": True}

"""認證代理（關鍵架構）：前端不直連 supabase.co，全部 auth 請求經後端轉發。

原因：部分地區網絡無法直達 *.supabase.co（實測本網絡連接被重置）。
後端部署於美區（Render），可正常訪問 Supabase Auth API。
"""
from __future__ import annotations

import httpx
from fastapi import APIRouter, HTTPException, Request
from pydantic import BaseModel

from app.config import settings

router = APIRouter(prefix="/auth", tags=["auth"])

_SB = settings.supabase_url.rstrip("/")
```

```
def _anon_key() -> str:
    return settings.supabase_anon_key


def _anon_headers() -> dict:
    k = _anon_key()
    return {"apikey": k, "Authorization": f"Bearer {k}", "Content-Type": "application/json"}


class Credentials(BaseModel):
    email: str
    password: str


class MagicLinkRequest(BaseModel):
    email: str


class RefreshRequest(BaseModel):
    refresh_token: str


@router.post("/signup")
def signup(payload: Credentials) -> dict:
    try:
        resp = httpx.post(
            f"{_SB}/auth/v1/signup", headers=_anon_headers(),
            json={"email": payload.email, "password": payload.password}, timeout=20,
        )
    except httpx.HTTPError as exc:
        raise HTTPException(502, f"無法連接 Supabase Auth（服務端網絡問題）：{exc}") from exc
    if resp.status_code >= 400:
        raise HTTPException(resp.status_code, _err(resp))
    data = resp.json()
    if not data.get("session"):
        return {"status": "email_confirmation_required", "user": data.get("user")}
    return {"status": "ok", "session": data["session"], "user": data["user"]}


@router.post("/signin")
def signin(payload: Credentials) -> dict:
    try:
        resp = httpx.post(
            f"{_SB}/auth/v1/token?grant_type=password", headers=_anon_headers(),
            json={"email": payload.email, "password": payload.password}, timeout=20,
        )
    except httpx.HTTPError as exc:
        raise HTTPException(502, f"無法連接 Supabase Auth（服務端網絡問題）：{exc}") from exc
    if resp.status_code >= 400:
        raise HTTPException(resp.status_code, _err(resp))
    return resp.json()


@router.post("/magic-link")
def magic_link(payload: MagicLinkRequest) -> dict:
    try:
        resp = httpx.post(
            f"{_SB}/auth/v1/magiclink", headers=_anon_headers(),
            json={"email": payload.email, "emailRedirectTo": settings.frontend_url}, timeout=20,
        )
    except httpx.HTTPError as exc:
        raise HTTPException(502, f"無法連接 Supabase Auth（服務端網絡問題）：{exc}") from exc
    if resp.status_code >= 400:
```

```
        raise HTTPException(resp.status_code, _err(resp))
    return {"status": "sent"}


@router.post("/refresh")
def refresh(payload: RefreshRequest) -> dict:
    try:
        resp = httpx.post(
            f"{_SB}/auth/v1/token?grant_type=refresh_token", headers=_anon_headers(),
            json={"refresh_token": payload.refresh_token}, timeout=20,
        )
    except httpx.HTTPError as exc:
        raise HTTPException(502, f"無法連接 Supabase Auth（服務端網絡問題）: {exc}") from exc
    if resp.status_code >= 400:
        raise HTTPException(resp.status_code, _err(resp))
    return resp.json()


class AdminCreate(BaseModel):
    email: str
    password: str
    secret: str


@router.post("/admin-create")
def admin_create(payload: AdminCreate) -> dict:
    """維護端點：用 service_role 走 GoTrue 管理 API 正規創建用戶（含郵箱確認）。

  僅供管理員運維使用，secret 由 OPS_SECRET 環境變量保護。
    """
    if not settings.ops_secret or payload.secret != settings.ops_secret:
        raise HTTPException(403, "forbidden")
    try:
        resp = httpx.post(
            f"{_SB}/auth/v1/admin/users",
            headers={
                "apikey": settings.supabase_service_role_key,
                "Authorization": f"Bearer {settings.supabase_service_role_key}",
                "Content-Type": "application/json",
            },
            json={"email": payload.email, "password": payload.password, "email_confirm": True},
            timeout=25,
        )
    except httpx.HTTPError as exc:
        raise HTTPException(502, f"無法連接 Supabase Auth: {exc}") from exc
    try:
        body = resp.json()
    except Exception:  # noqa: BLE001
        body = resp.text[:300]
    return {"status": resp.status_code, "body": body}


def _err(resp: httpx.Response) -> str:
    try:
        return resp.json().get("msg") or resp.json().get("error_description") or resp.text[:300]
    except Exception:  # noqa: BLE001
        return resp.text[:300]

class ChangePasswordRequest(BaseModel):
    old_password: str
    new_password: str


@router.post("/change-password")
```

```
def change_password(user: CurrentUser, db: DB, payload: ChangePasswordRequest, request: Request)
-> dict:
    """修改密碼：先驗證舊密碼，再經 GoTrue 更新（配合前端 E2EE 密鑰遷移）。"""
    import httpx as _httpx

    if len(payload.new_password) < 6:
        raise HTTPException(422, "新密碼至少 6 位")
    email = user.get("email")
    # 驗證舊密碼
    try:
        r = _httpx.post(
            f"{settings.supabase_url}/auth/v1/token?grant_type=password",
            headers={"apikey": settings.supabase_anon_key},
            json={"email": email, "password": payload.old_password},
            timeout=20,
        )
    except Exception as exc:  # noqa: BLE001
        raise HTTPException(502, f"驗證服務不可用: {exc}") from exc
    if r.status_code != 200:
        raise HTTPException(401, "舊密碼不正確")
    # 用當前用戶 token 更新密碼
    token = request.headers.get("Authorization", "").removeprefix("Bearer ").strip()
    r = _httpx.put(
        f"{settings.supabase_url}/auth/v1/user",
        headers={"apikey": settings.supabase_anon_key, "Authorization": f"Bearer {token}"},
        json={"password": payload.new_password},
        timeout=20,
    )
    if r.status_code != 200:
        raise HTTPException(502, f"密碼更新失敗: {r.text[:200]}")
    return {"ok": True, "note": "密碼已更新；敏感字段密文已由前端用新密鑰重新加密"}

"""關注公司管理 + 公司招聘官網自動發現。"""
from __future__ import annotations

from typing import Annotated

from fastapi import APIRouter, Depends, HTTPException, Request
from pydantic import BaseModel
from sqlalchemy import text
from sqlalchemy.orm import Session

from app.core.audit import log_audit
from app.core.auth import get_current_user
from app.core.jsonutil import j
from app.db import get_db
from app.schemas.accounts import PLATFORMS, WatchedCompanyCreate, WatchedCompanyUpdate

router = APIRouter(prefix="/companies", tags=["companies"])

CurrentUser = Annotated[dict, Depends(get_current_user)]
DB = Annotated[Session, Depends(get_db)]


@router.get("")
def list_companies(user: CurrentUser, db: DB) -> list[dict]:
    rows = db.execute(
        text(
            """
            SELECT DISTINCT ON (COALESCE(careers_url, company_name)) id, company_name, aliases,
                   careers_url, platform, notes, is_active, created_at
            FROM watched_companies WHERE user_id = :uid
            ORDER BY COALESCE(careers_url, company_name), created_at
            """
        ),
```

```
        {"uid": user["id"]},
    ).mappings().all()
    return [
        {
            "id": str(r["id"]),
            "company_name": r["company_name"],
            "aliases": r["aliases"] or [],
            "careers_url": r["careers_url"],
            "platform": r["platform"],
            "notes": r["notes"],
            "is_active": r["is_active"],
            "created_at": r["created_at"].isoformat(),
        }
        for r in rows
    ]


@router.post("", status_code=201)
def create_company(
    user: CurrentUser, db: DB, payload: WatchedCompanyCreate, request: Request
) -> dict:
    if payload.platform not in PLATFORMS:
        raise HTTPException(422, f"platform 必须是{PLATFORMS} 之一")
    row = db.execute(
        text(
            """
            INSERT INTO watched_companies
              (user_id, company_name, aliases, careers_url, platform, notes)
            VALUES (:uid, :name, CAST(:aliases AS jsonb), :url, :platform, :notes)
            RETURNING id
            """
        ),
        {
            "uid": user["id"],
            "name": payload.company_name,
            "aliases": j(payload.aliases or []),
            "url": payload.careers_url,
            "platform": payload.platform,
            "notes": payload.notes,
        },
    ).mappings().one()
    db.commit()
    log_audit(db, user["id"], "company.create", "watched_companies", str(row["id"]),
              {"name": payload.company_name}, request.client.host if request.client else None)
    return {"id": str(row["id"])}


@router.patch("/{company_id}")
def update_company(
    user: CurrentUser, db: DB, company_id: str, payload: WatchedCompanyUpdate
) -> dict:
    fields = payload.model_dump(exclude_none=True)
    if "aliases" in fields:
        fields["aliases"] = j(fields["aliases"] or [])
    sets = ", ".join(f"{k} = CAST(:{k} AS jsonb)" if k == "aliases" else f"{k} = :{k}"
                     for k in fields)
    result = db.execute(
        text(
            f"UPDATE watched_companies SET {sets}, updated_at = now() "
            "WHERE id = :cid AND user_id = :uid RETURNING id"
        ) if sets else text("SELECT id FROM watched_companies WHERE id = :cid AND user_id
= :uid"),
        {"cid": company_id, "uid": user["id"], **fields},
    )
    db.commit()
```

```
    if not result.first():
        raise HTTPException(404, "公司不存在")
    return {"ok": True}


@router.delete("/{company_id}", status_code=204)
def delete_company(user: CurrentUser, db: DB, company_id: str) -> None:
    row = db.execute(
        text("DELETE FROM watched_companies WHERE id = :cid AND user_id = :uid RETURNING id"),
        {"cid": company_id, "uid": user["id"]},
    ).first()
    if not row:
        raise HTTPException(404, "公司不存在")
    db.commit()


class DiscoverRequest(BaseModel):
    name: str


@router.post("/discover")
def discover_company(user: CurrentUser, payload: DiscoverRequest) -> dict:
    """按公司名自動發現招聘官網：LLM 給候選 + 實網探測驗證。"""
    import re

    import httpx as _httpx

    from app.core.llm import chat_json

    name = payload.name.strip()[:100]
    out = chat_json(
        "你熟悉各公司招聘官網。給出公司招聘官網地址候選。輸出 JSON: "
        '{"candidates": ["url1", "url2", ...], "careers_hint": "一句話說明"}',
        f"公司名：{name}",
        max_tokens=500,
    )
    verified = []
    for url in (out.get("candidates") or [])[:6]:
        if not url.startswith("http"):
            continue
        try:
            r = _httpx.get(url, timeout=12, follow_redirects=True,
                           headers={"User-Agent": "Mozilla/5.0"})
            if r.status_code == 200:
                m = re.search(r"<title>([^<]*)</title>", r.text, re.I)
                verified.append({"url": str(r.url), "status": 200,
                                 "title": (m.group(1).strip()[:80] if m else None)})
            else:
                verified.append({"url": url, "status": r.status_code, "title": None})
        except Exception as exc:  # noqa: BLE001
            verified.append({"url": url, "status": "unreachable", "title": str(exc)[:80]})
    return {"name": name, "hint": out.get("careers_hint"), "candidates": verified}

"""文件上傳與管理（FR-1b）。附件入 R2，DB 只存元數據（GAP-8）。"""
from __future__ import annotations

import uuid
from typing import Annotated

from fastapi import APIRouter, BackgroundTasks, Depends, File, HTTPException, Request,
UploadFile
from sqlalchemy import text
from sqlalchemy.orm import Session

from app.config import settings
```

```
from app.core.audit import log_audit
from app.core.auth import get_current_user
from app.core.r2 import object_url, presigned_url, upload_bytes
from app.db import get_db
from app.services.parse_service import parse_document

router = APIRouter(prefix="/documents", tags=["documents"])

CurrentUser = Annotated[dict, Depends(get_current_user)]
DB = Annotated[Session, Depends(get_db)]

MAX_BYTES = 50 * 1024 * 1024  # Supabase Storage 免費層單文件上限
DOC_TYPES = ("resume", "cover_letter", "transcript", "visa", "certificate", "portfolio",
"other")
ALLOWED_MIME = {
    "application/pdf": "pdf",
    "application/msword": "doc",
    "application/vnd.openxmlformats-officedocument.wordprocessingml.document": "docx",
    "text/plain": "txt",
    "image/jpeg": "jpg",
    "image/png": "png",
    "image/webp": "webp",
}


@router.post("", status_code=201)
async def upload_document(
    user: CurrentUser,
    db: DB,
    request: Request,
    doc_type: str,
    file: UploadFile = File(...),
    lang: str = "zh",
) -> dict:
    if doc_type not in DOC_TYPES:
        raise HTTPException(422, f"doc_type 必須是 {DOC_TYPES} 之一")
    if lang not in ("zh", "en"):
        raise HTTPException(422, "lang 必須是 zh 或 en")
    if file.content_type not in ALLOWED_MIME:
        raise HTTPException(422, f"不支持的文件類型: {file.content_type}")
    data = await file.read()
    if len(data) > MAX_BYTES:
        raise HTTPException(413, "文件超過 50MB 上限")

    doc_id = uuid.uuid4()
    ext = ALLOWED_MIME[file.content_type]
    storage_path = f"users/{user['id']}/{doc_id}.{ext}"
    upload_bytes(storage_path, data, file.content_type)

    row = db.execute(
        text(
            """
            INSERT INTO user_documents
              (user_id, doc_type, file_name, storage_path, mime, size_bytes, parse_status, lang)
            VALUES (:uid, :doc_type, :name, :path, :mime, :size, 'pending', :lang)
            RETURNING id
            """
        ),
        {
            "uid": user["id"],
            "doc_type": doc_type,
            "name": file.filename or f"{doc_id}.{ext}",
            "path": storage_path,
            "mime": file.content_type,
            "size": len(data),
```

```
            "lang": lang,
        },
    ).mappings().one()
    db.commit()
    log_audit(db, user["id"], "document.upload", "user_documents", str(row["id"]),
              {"doc_type": doc_type, "size": len(data), "lang": lang},
              request.client.host if request.client else None)
    return {"id": str(row["id"])}


@router.get("")
def list_documents(
    user: CurrentUser, db: DB, doc_type: str | None = None,
    limit: int = 20, offset: int = 0,
) -> dict:
    limit = min(max(limit, 1), 100)
    rows = db.execute(
        text(
            """
            SELECT id, doc_type, file_name, mime, size_bytes, parse_status, created_at,
is_active, lang
            FROM user_documents
            WHERE user_id = :uid AND deleted_at IS NULL
              AND (CAST(:t AS text) IS NULL OR CAST(doc_type AS text) = :t)
            ORDER BY created_at DESC
            LIMIT :lim OFFSET :off
            """
        ),
        {"uid": user["id"], "t": doc_type, "lim": limit, "off": offset},
    ).mappings().all()
    total = db.execute(
        text(
            """
            SELECT COUNT(*) AS n FROM user_documents
            WHERE user_id = :uid AND deleted_at IS NULL
              AND (CAST(:t AS text) IS NULL OR CAST(doc_type AS text) = :t)
            """
        ),
        {"uid": user["id"], "t": doc_type},
    ).mappings().one()["n"]
    return {
        "items": [
            {
                "id": str(r["id"]),
                "doc_type": r["doc_type"],
                "file_name": r["file_name"],
                "mime": r["mime"],
                "size_bytes": r["size_bytes"],
                "parse_status": r["parse_status"],
                "is_active": r["is_active"],
                "lang": r["lang"],
                "created_at": r["created_at"].isoformat(),
            }
            for r in rows
        ],
        "total": total,
        "limit": limit,
        "offset": offset,
    }


@router.post("/{doc_id}/parse", status_code=202)
def trigger_parse(
    user: CurrentUser, db: DB, doc_id: str, background_tasks: BackgroundTasks
) -> dict:
```

```
    """觸發文檔解析（文本抽取 + LLM 結構化），同步執行；解析成功後後台重新匹配。"""
    row = db.execute(
        text(
            "SELECT storage_path, mime FROM user_documents "
            "WHERE id = :did AND user_id = :uid AND deleted_at IS NULL"
        ),
        {"did": doc_id, "uid": user["id"]},
    ).mappings().first()
    if not row:
        raise HTTPException(404, "文件不存在")
    try:
        parse_document(db, doc_id, user["id"], row["storage_path"], row["mime"])
    except Exception as exc:  # noqa: BLE001
        raise HTTPException(500, f"解析失敗: {exc}") from exc
    # 簡歷內容變化 → 後台重新匹配（畫像更新後會再觸發一次完整重匹配）
    from app.api.profile import _rematch_bg

    background_tasks.add_task(_rematch_bg, user["id"])
    return {"ok": True, "note": "解析完成，正在後台重新匹配崗位"}


@router.get("/{doc_id}")
def get_document(user: CurrentUser, db: DB, doc_id: str) -> dict:
    row = db.execute(
        text(
            "SELECT storage_path, file_name, mime, doc_type, parse_status "
            "FROM user_documents WHERE id = :did AND user_id = :uid AND deleted_at IS NULL"
        ),
        {"did": doc_id, "uid": user["id"]},
    ).mappings().first()
    if not row:
        raise HTTPException(404, "文件不存在")
    return {
        "id": doc_id,
        "file_name": row["file_name"],
        "doc_type": row["doc_type"],
        "mime": row["mime"],
        "parse_status": row["parse_status"],
        "url": object_url(row["storage_path"]),
    }


@router.delete("/{doc_id}", status_code=204)
def delete_document(user: CurrentUser, db: DB, doc_id: str, request: Request) -> None:
    row = db.execute(
        text(
            "UPDATE user_documents SET deleted_at = now(), is_active = false "
            "WHERE id = :did AND user_id = :uid AND deleted_at IS NULL RETURNING id"
        ),
        {"did": doc_id, "uid": user["id"]},
    ).first()
    if not row:
        raise HTTPException(404, "文件不存在")
    db.commit()
    log_audit(db, user["id"], "document.delete", "user_documents", doc_id,
              {}, request.client.host if request.client else None)

"""崗位瀏覽與手動採集端點（FR-3a）。"""
from __future__ import annotations

import logging
from typing import Annotated

from fastapi import APIRouter, BackgroundTasks, Depends, HTTPException
from pydantic import BaseModel
```

```
from sqlalchemy import text
from sqlalchemy.orm import Session

from app.core.auth import get_current_user
from app.core.jsonutil import j
from app.db import get_db

logger = logging.getLogger(__name__)

router = APIRouter(prefix="/jobs", tags=["jobs"])

CurrentUser = Annotated[dict, Depends(get_current_user)]
DB = Annotated[Session, Depends(get_db)]


class CollectRequest(BaseModel):
    company: str | None = None


@router.get("")
def list_jobs(
    user: CurrentUser, db: DB, q: str | None = None, company: str | None = None,
    campus: str | None = None, org: str | None = None,
    limit: int = 20, offset: int = 0,
) -> dict:
    limit = min(max(limit, 1), 200)
    rows = db.execute(
        text(
            """
            SELECT p.id, p.company_name, p.title, p.location, p.department, p.source,
                   p.apply_url, p.first_seen_at, p.posted_at, p.is_campus, p.org_level,
                   m.match_score, m.priority_score, m.status AS match_status, m.id AS match_id
            FROM job_postings p
            LEFT JOIN job_matches m ON m.posting_id = p.id AND m.user_id = :uid
            WHERE p.is_active = true
              AND (CAST(:q AS text) IS NULL OR p.title ILIKE '%'||:q||'%' OR p.description_text
ILIKE '%'||:q||'%')
              AND (CAST(:co AS text) IS NULL OR p.company_name ILIKE '%'||:co||'%')
              AND (CAST(:ca AS text) IS NULL OR (CAST(:ca AS text)='campus' AND p.is_campus)
                   OR (CAST(:ca AS text)='society' AND NOT p.is_campus))
              AND (CAST(:og AS text) IS NULL OR p.org_level = :og)
            ORDER BY p.first_seen_at DESC
            LIMIT :lim OFFSET :off
            """
        ),
        {"uid": user["id"], "q": q, "co": company, "ca": campus, "og": org,
         "lim": limit, "off": offset},
    ).mappings().all()
    total = db.execute(
        text(
            """
            SELECT COUNT(*) AS n FROM job_postings p
            WHERE p.is_active = true
              AND (CAST(:q AS text) IS NULL OR p.title ILIKE '%'||:q||'%' OR p.description_text
ILIKE '%'||:q||'%')
              AND (CAST(:co AS text) IS NULL OR p.company_name ILIKE '%'||:co||'%')
              AND (CAST(:ca AS text) IS NULL OR (CAST(:ca AS text)='campus' AND p.is_campus)
                   OR (CAST(:ca AS text)='society' AND NOT p.is_campus))
              AND (CAST(:og AS text) IS NULL OR p.org_level = :og)
            """
        ),
        {"q": q, "co": company, "ca": campus, "og": org},
    ).mappings().one()["n"]
    return {
        "items": [
```

```
            {
                "id": str(r["id"]),
                "company_name": r["company_name"],
                "title": r["title"],
                "location": r["location"],
                "department": r["department"],
                "source": r["source"],
                "apply_url": r["apply_url"],
                "first_seen_at": r["first_seen_at"].isoformat(),
                "posted_at": r["posted_at"].isoformat() if r["posted_at"] else None,
                "match_score": r["match_score"],
                "priority_score": r["priority_score"],
                "is_campus": r["is_campus"],
                "org_level": r["org_level"],
                "match_status": r["match_status"],
                "match_id": str(r["match_id"]) if r["match_id"] else None,
            }
            for r in rows
        ],
        "limit": limit,
        "offset": offset,
        "total": total,
    }


@router.get("/collect-status")
def collect_status(user: CurrentUser, db: DB) -> dict:
    """採集進度：前台輪詢此端點，running=false 且 finished_at 有值即完成。"""
    row = db.execute(
        text("SELECT collect_state FROM user_settings WHERE user_id=:u"),
        {"u": user["id"]},
    ).mappings().first()
    return dict(row["collect_state"] or {}) if row and row["collect_state"] else {"running":
False}


@router.get("/{posting_id}")
def get_job(user: CurrentUser, db: DB, posting_id: str) -> dict:
    row = db.execute(
        text(
            """
            SELECT p.*, m.match_score, m.reasons, m.hard_failures, m.status AS match_status
            FROM job_postings p
            LEFT JOIN job_matches m ON m.posting_id = p.id AND m.user_id = :uid
            WHERE p.id = :pid
            """
        ),
        {"uid": user["id"], "pid": posting_id},
    ).mappings().first()
    if not row:
        raise HTTPException(404, "崗位不存在")
    return {
        "id": str(row["id"]),
        "company_name": row["company_name"],
        "title": row["title"],
        "location": row["location"],
        "department": row["department"],
        "source": row["source"],
        "apply_url": row["apply_url"],
        "description_text": row["description_text"],
        "first_seen_at": row["first_seen_at"].isoformat(),
        "match_score": row["match_score"],
        "match_status": row["match_status"],
        "reasons": row["reasons"] or [],
        "hard_failures": row["hard_failures"] or [],
```

```
    }


@router.post("/collect/run", status_code=202)
def run_collect(user: CurrentUser, db: DB, payload: CollectRequest, background_tasks:
BackgroundTasks) -> dict:
    """異步觸發：全量或按公司採集 + 匹配（後台執行）。"""
    from app.collectors.pipeline import collect_for_user

    uid = user["id"]
    company = (payload.company or "").strip()

    def _run() -> None:
        import datetime as _dt

        from app.db import SessionLocal

        s = SessionLocal()
        s.execute(
            text(
                "INSERT INTO user_settings (user_id, collect_state) VALUES (:u, CAST(:st AS
jsonb)) "
                "ON CONFLICT (user_id) DO UPDATE SET collect_state=CAST(:st AS jsonb),
updated_at=now()"
            ),
            {"u": uid, "st": j({"running": True, "finished_at": None, "report": None})},
        )
        s.commit()
        try:
            report = collect_for_user(s, uid, company_name=company or None)
            s.execute(
                text(
                    "UPDATE user_settings SET collect_state=CAST(:st AS jsonb), updated_at=now()
"
                    "WHERE user_id=:u"
                ),
                {"u": uid, "st": j({
                    "running": False,
                    "finished_at": _dt.datetime.now(_dt.timezone.utc).isoformat(),
                    "report": {k: report.get(k) for k in ("new", "updated", "matched", "errors",
"collected")},
                })},
            )
            s.commit()
        except Exception:  # noqa: BLE001
            logger.exception("後台採集失敗 user=%s", uid)
            s.execute(
                text(
                    "UPDATE user_settings SET collect_state=CAST(:st AS jsonb), updated_at=now()
"
                    "WHERE user_id=:u"
                ),
                {"u": uid, "st": j({"running": False, "finished_at": None, "report": {"errors":
1}})},
            )
            s.commit()
        finally:
            s.close()

    background_tasks.add_task(_run)
    return {"started": True, "note": ("僅採集該公司" if company else "全量採集") + "，後台執行中"}


@router.get("/collect-status")
def collect_status(user: CurrentUser, db: DB) -> dict:
```

```
    """採集進度：前台輪詢此端點，running=false 且 finished_at 有值即完成。"""
    row = db.execute(
        text("SELECT collect_state FROM user_settings WHERE user_id=:u"),
        {"u": user["id"]},
    ).mappings().first()
    return dict(row["collect_state"] or {}) if row and row["collect_state"] else {"running":
False}

"""崗位匹配端點（FR-1f）。採集流水線（P3）也復用 score_posting。"""
from __future__ import annotations

import logging
from typing import Annotated

from fastapi import APIRouter, BackgroundTasks, Depends, HTTPException
from pydantic import BaseModel
from sqlalchemy import text
from sqlalchemy.orm import Session

from app.core.auth import get_current_user
from app.core.jsonutil import loads_json
from app.db import get_db
from app.services.matching import score_posting

logger = logging.getLogger(__name__)

router = APIRouter(prefix="/matches", tags=["matches"])

CurrentUser = Annotated[dict, Depends(get_current_user)]
DB = Annotated[Session, Depends(get_db)]

REMATCH_LIMIT = 200  # 每輪評分上限（控制 LLM 成本）


def rematch_all_for_user(db: Session, user_id: str, limit: int = 30) -> dict:
    """用當前畫像對用戶匹配做一輪評分（補缺失優先 + 重算最舊的），返回進度。

  準確性 100% 策略：分批多輪直到 remaining=0；
  每輪優先補評「完全沒有 match 行」的崗位，再重算最舊的已有 match。
    """
    total = 0
    # 1) 補評缺失 match 的崗位（採集時漏評/新公司）
    missing = db.execute(
        text(
            """
            SELECT p.id FROM job_postings p
            LEFT JOIN job_matches m ON m.posting_id = p.id AND m.user_id = :u
            WHERE m.id IS NULL AND p.is_active = true
            ORDER BY p.first_seen_at DESC LIMIT :n
            """
        ),
        {"u": user_id, "n": limit},
    ).mappings().all()
    for r in missing:
        try:
            score_posting(db, user_id, str(r["id"]))
            total += 1
        except Exception:  # noqa: BLE001
            logger.exception("補評失敗 user=%s posting=%s", user_id, r["id"])
    # 2) 重算最舊的已有 match
    if len(missing) < limit:
        stale = db.execute(
            text(
                "SELECT posting_id FROM job_matches WHERE user_id=:u "
                "ORDER BY updated_at ASC LIMIT :n"
```

```
            ),
            {"u": user_id, "n": limit - len(missing)},
        ).mappings().all()
        for r in stale:
            try:
                score_posting(db, user_id, str(r["posting_id"]))
                total += 1
            except Exception:  # noqa: BLE001
                logger.exception("重新匹配失敗 user=%s posting=%s", user_id, r["posting_id"])
    # 3) 剩餘量（供進度查詢/多輪調度）
    remaining = db.execute(
        text(
            """
            SELECT (SELECT COUNT(*) FROM job_postings p
                    LEFT JOIN job_matches m ON m.posting_id = p.id AND m.user_id = :u
                    WHERE m.id IS NULL AND p.is_active = true) AS miss,
                   (SELECT COUNT(*) FROM job_matches WHERE user_id = :u) AS has
            """
        ),
        {"u": user_id},
    ).mappings().one()
    return {"scored": total, "remaining_missing": int(remaining["miss"]),
            "total_matches": int(remaining["has"])}


@router.post("/{posting_id}/score", status_code=201)
def score(user: CurrentUser, db: DB, posting_id: str) -> dict:
    try:
        return score_posting(db, user["id"], posting_id)
    except ValueError as exc:
        raise HTTPException(400, str(exc)) from exc


@router.get("/companies")
def list_companies(user: CurrentUser, db: DB) -> list[dict]:
    """匹配頁公司篩選列表（公司 + 匹配數 + 最高分；含 hard_failed 的公司）。"""
    rows = db.execute(
        text(
            """
            SELECT p.company_name, COUNT(*) AS n, MAX(m.match_score) AS top
            FROM job_matches m JOIN job_postings p ON p.id = m.posting_id
            WHERE m.user_id = :uid
            GROUP BY p.company_name ORDER BY top DESC NULLS LAST, n DESC LIMIT 100
            """
        ),
        {"uid": user["id"]},
    ).mappings().all()
    return [
        {"company_name": r["company_name"], "count": r["n"], "top_score": r["top"]}
        for r in rows
    ]


def rematch_company(db: Session, user_id: str, company: str, batch: int = 50) -> dict:
    """按公司重新匹配：分批循環直到該公司全部崗位重算。"""
    total = 0
    for _ in range(20):  # 每輪 50 個，最多 1000 個/公司
        rows = db.execute(
            text(
                """
                SELECT p.id FROM job_postings p
                WHERE p.is_active = true AND p.company_name ILIKE '%'||:co||'%'
                ORDER BY p.first_seen_at DESC LIMIT :n OFFSET :off
                """
            ),
```

```
            {"co": company, "n": batch, "off": total},
        ).mappings().all()
        if not rows:
            break
        for r in rows:
            try:
                score_posting(db, user_id, str(r["id"]))
                total += 1
            except Exception:  # noqa: BLE001
                logger.exception("按公司匹配失敗 user=%s posting=%s", user_id, r["id"])
    return {"scored": total, "company": company, "remaining_missing": 0, "total_matches": total}


def rematch_full(db: Session, user_id: str, max_rounds: int = 30, batch: int = 30) -> dict:
    """分批多輪直到全量補評完成。返回最終進度。"""
    res = {"scored": 0, "remaining_missing": -1, "total_matches": 0}
    for _round in range(max_rounds):
        res = rematch_all_for_user(db, user_id, limit=batch)
        if res["remaining_missing"] == 0:
            break
    return res


class RematchRequest(BaseModel):
    company: str | None = None


@router.post("/rematch", status_code=202)
def rematch(user: CurrentUser, db: DB, payload: RematchRequest, background_tasks:
BackgroundTasks) -> dict:
    """重新匹配：全量或按公司（補評缺失 + 重算），後台執行。"""
    uid = user["id"]
    company = (payload.company or "").strip()

    def _run() -> None:
        from app.db import SessionLocal

        s = SessionLocal()
        try:
            if company:
                res = rematch_company(s, uid, company)
            else:
                res = rematch_full(s, uid)
            logger.info("重新匹配完成 user=%s %s", uid, res)
        except Exception:  # noqa: BLE001
            logger.exception("重新匹配失敗 user=%s", uid)
        finally:
            s.close()

    background_tasks.add_task(_run)
    return {"started": True, "note": ("僅重新匹配該公司" if company else "全量重新匹配") + "，後台執行
中"}


@router.get("/rematch-status")
def rematch_status(user: CurrentUser, db: DB) -> dict:
    """重新匹配進度：未評分崗位數 + 已有匹配數。"""
    row = db.execute(
        text(
            """
            SELECT (SELECT COUNT(*) FROM job_postings p
                    LEFT JOIN job_matches m ON m.posting_id = p.id AND m.user_id = :u
                    WHERE m.id IS NULL AND p.is_active = true) AS miss,
                   (SELECT COUNT(*) FROM job_matches WHERE user_id = :u) AS has
            """
```

```
        ),
        {"u": user["id"]},
    ).mappings().one()
    return {"remaining_missing": int(row["miss"]), "total_matches": int(row["has"])}


@router.get("")
def list_matches(
    user: CurrentUser, db: DB, status: str | None = None,
    company: str | None = None,
    limit: int = 20, offset: int = 0,
    campus: str | None = None, org: str | None = None,
) -> dict:
    limit = min(max(limit, 1), 100)
    rows = db.execute(
        text(
            """
            SELECT m.id, m.posting_id, m.match_score, m.priority_score, m.hard_failures,
                   m.reasons, m.score_breakdown, m.direction_scores, m.status,
                   p.company_name, p.title, p.location, p.department, p.is_campus,
                   p.org_level, p.apply_url, p.first_seen_at, p.raw->'directions' AS directions
            FROM job_matches m
            JOIN job_postings p ON p.id = m.posting_id
            WHERE m.user_id = :uid AND (CAST(:st AS text) IS NULL OR CAST(m.status AS text)
= :st)
              AND (CAST(:co AS text) IS NULL OR p.company_name ILIKE '%'||:co||'%')
              AND (CAST(:ca AS text) IS NULL OR (CAST(:ca AS text)='campus' AND p.is_campus)
                   OR (CAST(:ca AS text)='society' AND NOT p.is_campus))
              AND (CAST(:og AS text) IS NULL OR p.org_level = :og)
            ORDER BY m.priority_score DESC NULLS LAST, m.match_score DESC NULLS LAST,
                     p.first_seen_at DESC
            LIMIT :lim OFFSET :off
            """
        ),
        {"uid": user["id"], "st": status, "co": company, "ca": campus, "og": org,
         "lim": limit, "off": offset},
    ).mappings().all()
    total = db.execute(
        text(
            """
            SELECT COUNT(*) AS n FROM job_matches m
            JOIN job_postings p ON p.id = m.posting_id
            WHERE m.user_id = :uid AND (CAST(:st AS text) IS NULL OR CAST(m.status AS text)
= :st)
              AND (CAST(:co AS text) IS NULL OR p.company_name ILIKE '%'||:co||'%')
              AND (CAST(:ca AS text) IS NULL OR (CAST(:ca AS text)='campus' AND p.is_campus)
                   OR (CAST(:ca AS text)='society' AND NOT p.is_campus))
              AND (CAST(:og AS text) IS NULL OR p.org_level = :og)
            """
        ),
        {"uid": user["id"], "st": status, "co": company, "ca": campus, "og": org},
    ).mappings().one()["n"]
    return {
        "items": [
            {
                "id": str(r["id"]),
                "posting_id": str(r["posting_id"]),
                "match_score": r["match_score"],
                "priority_score": r["priority_score"],
                "is_campus": r["is_campus"],
                "org_level": r["org_level"],
                "department": r["department"],
                "directions": loads_json(r["directions"])
                if isinstance(r["directions"], str) else (r["directions"] or []),
                "hard_failures": r["hard_failures"] or [],
```

```
                "score_breakdown": r["score_breakdown"] or {},
                "direction_scores": loads_json(r["direction_scores"])
                if isinstance(r["direction_scores"], str)
                else (r["direction_scores"] or []),
                "reasons": r["reasons"] or [],
                "status": r["status"],
                "company_name": r["company_name"],
                "title": r["title"],
                "location": r["location"],
                "apply_url": r["apply_url"],
                "first_seen_at": r["first_seen_at"].isoformat(),
            }
            for r in rows
        ],
        "total": total,
        "limit": limit,
        "offset": offset,
    }

"""個人資料與設置端點（FR-1a/1c/1f/1g/1h）。

全部端點經 get_current_user 注入當前用戶，多租戶隔離雙重保證：
1) 代碼層強制 user_id = 當前用戶
2) 數據庫層 RLS（見 alembic/versions/0001_initial.py）
"""
from __future__ import annotations

from typing import Annotated

from fastapi import APIRouter, Depends, HTTPException
from sqlalchemy import text
from sqlalchemy.orm import Session

from app.core.auth import get_current_user
from app.core.jsonutil import j
from app.core.security import decrypt_field, encrypt_field, get_dek_for_user
from app.db import get_db
from app.schemas.settings import SettingsUpdate
from pydantic import BaseModel

router = APIRouter(tags=["me"])

CurrentUser = Annotated[dict, Depends(get_current_user)]
DB = Annotated[Session, Depends(get_db)]


class PushChannelUpdate(BaseModel):
    value: str | None = None


class HardFactUpdate(BaseModel):
    key: str
    value: str | None = None


HARD_FACT_KEYS = {
    "visa_status", "preferred_locations", "salary_expectation", "availability", "total_years",
}


@router.put("/me/hard-facts")
def update_hard_fact(user: CurrentUser, db: DB, payload: HardFactUpdate) -> dict:
    """更新硬性事實：同步寫入畫像 hard_facts 與元信息表（兩處保持一致）。"""
    if payload.key not in HARD_FACT_KEYS:
        raise HTTPException(422, f"key 必須是 {sorted(HARD_FACT_KEYS)} 之一")
```

```
    key, value = payload.key, (payload.value or "").strip()
    # 1) 畫像 hard_facts（Python 合併後 CAST 寫回，避免 jsonb_set 參數類型問題）
    row = db.execute(
        text("SELECT hard_facts FROM candidate_profiles WHERE user_id=:uid AND
is_current=true"),
        {"uid": user["id"]},
    ).mappings().first()
    if row is not None:
        hf = dict(row["hard_facts"] or {})
        hf[key] = value
        db.execute(
            text(
                "UPDATE candidate_profiles SET hard_facts=CAST(:hf AS jsonb) "
                "WHERE user_id=:uid AND is_current=true"
            ),
            {"hf": j(hf), "uid": user["id"]},
        )
    # 2) 元信息表同步
    db.execute(
        text(
            """
            INSERT INTO user_info_fields (user_id, key, value, source)
            VALUES (:uid, :k, :v, 'user')
            ON CONFLICT (user_id, key) DO UPDATE
              SET value=EXCLUDED.value, source='user', updated_at=now()
            """
        ),
        {"uid": user["id"], "k": key, "v": value},
    )
    db.commit()
    return {"ok": True}


class ProfileUpdate(BaseModel):
    full_name: str | None = None
    phone: str | None = None


@router.get("/me")
def read_me(user: CurrentUser) -> dict:
    return user


@router.patch("/me")
def update_me(user: CurrentUser, db: DB, payload: ProfileUpdate) -> dict:
    """更新個人資料（暱稱/電話）。"""
    if payload.full_name is not None:
        db.execute(
            text("UPDATE profiles SET full_name=:n, updated_at=now() WHERE id=:id"),
            {"n": payload.full_name.strip()[:64], "id": user["id"]},
        )
        user["full_name"] = payload.full_name.strip()[:64]
    if payload.phone is not None:
        db.execute(
            text("UPDATE profiles SET phone=:p, updated_at=now() WHERE id=:id"),
            {"p": payload.phone.strip()[:32], "id": user["id"]},
        )
        user["phone"] = payload.phone.strip()[:32]
    db.commit()
    return user


@router.get("/me/profile-full")
def profile_full(user: CurrentUser, db: DB) -> dict:
    """個人信息總覽：基本信息 + 畫像硬性事實/教育/經歷/技能 + 元信息表，供用戶檢查編輯。"""
```

```
    cp = db.execute(
        text(
            "SELECT hard_facts, education, experiences, skills, summary, generated_at "
            "FROM candidate_profiles WHERE user_id=:uid AND is_current=true "
            "ORDER BY generated_at DESC LIMIT 1"
        ),
        {"uid": user["id"]},
    ).mappings().first()
    fields = db.execute(
        text(
            "SELECT key, value, sensitive_enc, source, updated_at FROM user_info_fields "
            "WHERE user_id=:uid ORDER BY updated_at DESC LIMIT 200"
        ),
        {"uid": user["id"]},
    ).mappings().all()
    return {
        "basic": {
            "full_name": user.get("full_name"),
            "email": user.get("email"),
            "phone": user.get("phone"),
        },
        "profile": {
            "hard_facts": (cp["hard_facts"] if cp else None) or {},
            "education": (cp["education"] if cp else None) or [],
            "experiences": (cp["experiences"] if cp else None) or [],
            "skills": (cp["skills"] if cp else None) or [],
            "summary": (cp["summary"] if cp else None),
            "generated_at": cp["generated_at"].isoformat() if cp and cp["generated_at"] else
None,
        },
        "info_fields": [
            {"key": r["key"], "value": r["value"], "source": r["source"],
             "updated_at": r["updated_at"].isoformat(),
             "encrypted": r["sensitive_enc"]}
            for r in fields
        ],
    }


@router.get("/me/settings")
def read_settings(user: CurrentUser, db: DB) -> dict:
    row = db.execute(
        text(
            """
            INSERT INTO user_settings (user_id)
            VALUES (:uid)
            ON CONFLICT (user_id) DO NOTHING
            """
        ),
        {"uid": user["id"]},
    )
    db.commit()
    result = db.execute(
        text("SELECT * FROM user_settings WHERE user_id = :uid"),
        {"uid": user["id"]},
    ).mappings().one()
    return _row_to_dict(result)


@router.patch("/me/settings")
def update_settings(user: CurrentUser, db: DB, payload: SettingsUpdate) -> dict:
    fields = payload.model_dump(exclude_none=True)
    if "preferred_locations" in fields:
        fields["preferred_locations"] = j(fields["preferred_locations"] or [])
    if not fields:
```

```
        return read_settings(user, db)
    set_clause = ", ".join(f"{k} = :{k}" for k in fields)
    db.execute(
        text(
            f"""
            INSERT INTO user_settings (user_id)
            VALUES (:uid)
            ON CONFLICT (user_id) DO NOTHING
            """
        ),
        {"uid": user["id"]},
    )
    db.execute(
        text(f"UPDATE user_settings SET {set_clause}, updated_at = now() WHERE user_id = :uid"),
        {"uid": user["id"], **fields},
    )
    db.commit()
    return read_settings(user, db)


@router.get("/me/push-channels")
def read_push_channels(user: CurrentUser, db: DB) -> dict:
    """返回已配置渠道及掩碼值。"""
    row = db.execute(
        text("SELECT push_channels FROM user_settings WHERE user_id = :uid"),
        {"uid": user["id"]},
    ).mappings().first()
    channels = (row["push_channels"] if row else None) or {}
    result = {}
    for channel, key in (("pushplus", "pushplus_token_enc"),
                         ("wecom", "wecom_webhook_enc"), ("email", "email_enc")):
        enc = channels.get(key)
        if not enc:
            continue
        try:
            value = decrypt_field(get_dek_for_user(db, user["id"]), enc)
            result[channel] = _mask_value(channel, value)
        except Exception:  # noqa: BLE001
            result[channel] = "***（解密失敗，請重新配置）"
    return result


@router.patch("/me/push-channels")
def update_push_channel(
    user: CurrentUser, db: DB, channel: str, payload: PushChannelUpdate
) -> dict:
    """配置/清除推送渠道（FR-1h）。value 為 null 表示清除。"""
    value = payload.value
    if channel not in ("pushplus", "wecom", "email"):
        raise HTTPException(422, "channel 必須是 pushplus/wecom/email 之一")
    dek = get_dek_for_user(db, user["id"])
    db.execute(
        text(
            "INSERT INTO user_settings (user_id) VALUES (:uid) ON CONFLICT (user_id) DO NOTHING"
        ),
        {"uid": user["id"]},
    )
    key = {"pushplus": "pushplus_token_enc", "wecom": "wecom_webhook_enc", "email": "email_enc"}
[channel]
    if value is None:
        db.execute(
            text(
                "UPDATE user_settings SET push_channels = "
                "COALESCE(push_channels, '{}'::jsonb) - :key, updated_at = now() "
                "WHERE user_id = :uid"
```

```
            ),
            {"key": key, "uid": user["id"]},
        )
    else:
        enc = encrypt_field(dek, value)
        db.execute(
            text(
                "UPDATE user_settings SET push_channels = "
                "jsonb_set(COALESCE(push_channels, '{}'::jsonb), :key, to_jsonb(:enc::text)), "
                "updated_at = now() WHERE user_id = :uid"
            ),
            {"key": "{" + key + "}", "enc": enc, "uid": user["id"]},
        )
    db.commit()
    return read_push_channels(user, db)


def _mask_value(channel: str, value: str) -> str:
    if channel == "email":
        at = value.find("@")
        if at > 2:
            return value[:2] + "***" + value[at:]
        return "***@***"
    if len(value) <= 6:
        return "***"
    return value[:3] + "***" + value[-2:]


def _row_to_dict(row) -> dict:
    d = dict(row)
    d.pop("user_id", None)
    for k, v in d.items():
        if hasattr(v, "isoformat"):
            d[k] = v.isoformat()
    return d
```

```
"""人工接管任務端點（FR-3c：直接框 → 回填 → 繼續投遞）。"""
from __future__ import annotations

from typing import Annotated

from fastapi import APIRouter, Depends, HTTPException
from pydantic import BaseModel
from sqlalchemy import text
from sqlalchemy.orm import Session

from app.core.auth import get_current_user
from app.db import get_db
from app.services.apply_engine import answer_and_resume

router = APIRouter(prefix="/manual-tasks", tags=["manual-tasks"])

CurrentUser = Annotated[dict, Depends(get_current_user)]
DB = Annotated[Session, Depends(get_db)]


class TaskAnswer(BaseModel):
    answer: dict | None = None


@router.get("/{task_id}/fill-plan")
def task_fill_plan(user: CurrentUser, db: DB, task_id: str) -> dict:
    """RPA 輔助：為人工投遞生成「填寫方案」（按當前資料整理好的答案，可一鍵複製）。"""
    from app.services.apply_engine import _load_application
    from app.services.form_mapping import build_candidate_payload
```

```
    task = db.execute(
        text("SELECT application_id FROM manual_tasks WHERE id=:id AND user_id=:uid"),
        {"id": task_id, "uid": user["id"]},
    ).mappings().first()
    if not task:
        raise HTTPException(404, "任務不存在")
    app = _load_application(db, str(task["application_id"]))
    if not app:
        raise HTTPException(404, "投遞不存在")
    payload = build_candidate_payload(db, user["id"])
    skills = ", ".join(s["name"] for s in (payload.get("skills") or []))
    exp_lines = []
    for e in (payload.get("experiences") or [])[:5]:
        exp_lines.append(
            f"{e.get('start')} ~ {e.get('end')} {e.get('company')} {e.get('title')}: "
            + "; ".join((e.get("bullets") or [])[:2])
        )
    edu_lines = [
        f"{e.get('school')} {e.get('degree')} {e.get('major')}"
        for e in (payload.get("education") or [])
    ]
    fields = {
        "姓名": payload.get("full_name") or "（未填）",
        "手機": payload.get("phone") or "（未填）",
        "郵箱": payload.get("email") or "（未填）",
        "簽證狀態": payload.get("visa_status") or "（未填）",
        "期望薪資": payload.get("salary_expectation") or "（未填）",
        "可到崗時間": payload.get("availability") or "（未填）",
        "技能（複製到技能欄）": skills,
        "工作經歷（最多 5 段，每段兩行內）": "\n".join(exp_lines) or "（未填）",
        "教育（複製到教育欄）": "; ".join(edu_lines) or "（未填）",
    }
    text_block = "\n".join(f"{k}: {v}" for k, v in fields.items())
    # 機器可讀字段（RPA 擴展用）+ 簡歷文件（供 RPA 上傳）
    machine_fields = [
        {"key": "name", "label": "姓名", "value": payload.get("full_name") or ""},
        {"key": "phone", "label": "手機", "value": payload.get("phone") or ""},
        {"key": "email", "label": "郵箱", "value": payload.get("email") or ""},
        {"key": "visa", "label": "簽證", "value": payload.get("visa_status") or ""},
        {"key": "salary", "label": "薪資", "value": payload.get("salary_expectation") or ""},
        {"key": "availability", "label": "到崗", "value": payload.get("availability") or ""},
        {"key": "skills", "label": "技能", "value": skills},
        {"key": "experience", "label": "經歷", "value": "\n".join(exp_lines)},
        {"key": "education", "label": "教育", "value": "; ".join(edu_lines)},
    ]
    resume = None
    rf = payload.get("resume_file")
    if rf:
        from urllib.parse import unquote

        from app.core.r2 import object_url

        url = object_url(rf.get("storage_path"))
        resume = {"name": unquote(rf.get("name") or "resume.pdf"), "storage_path":
rf.get("storage_path"),
                  "download_url": url}
    return {"fields": fields, "copy_text": text_block, "machine_fields": machine_fields,
            "resume": resume, "company": app["company_name"], "title": app["title"],
            "apply_url": app.get("apply_url")}


@router.get("")
def list_tasks(user: CurrentUser, db: DB, status: str = "open") -> list[dict]:
    rows = db.execute(
```

```
        text(
            """
            SELECT t.id, t.task_type, t.question, t.options, t.status, t.created_at,
                   t.expires_at, a.status AS app_status,
                   p.company_name, p.title
            FROM manual_tasks t
            JOIN applications a ON a.id = t.application_id
            JOIN job_postings p ON p.id = a.posting_id
            WHERE t.user_id = :uid AND t.status = :st
            ORDER BY t.created_at DESC LIMIT 100
            """
        ),
        {"uid": user["id"], "st": status},
    ).mappings().all()
    return [
        {
            "id": str(r["id"]),
            "task_type": r["task_type"],
            "question": r["question"],
            "options": r["options"] or {},
            "status": r["status"],
            "created_at": r["created_at"].isoformat(),
            "expires_at": r["expires_at"].isoformat(),
            "application_status": r["app_status"],
            "company_name": r["company_name"],
            "title": r["title"],
        }
        for r in rows
    ]


@router.post("/{task_id}/answer", status_code=200)
def answer_task(user: CurrentUser, db: DB, task_id: str, payload: TaskAnswer) -> dict:
    result = answer_and_resume(db, task_id, user["id"], payload.answer)
    if result.get("error"):
        raise HTTPException(404, result["error"])
    return result


@router.post("/{task_id}/cancel", status_code=200)
def cancel_task(user: CurrentUser, db: DB, task_id: str) -> dict:
    row = db.execute(
        text(
            "UPDATE manual_tasks SET status='cancelled' "
            "WHERE id=:id AND user_id=:uid AND status='open' RETURNING application_id"
        ),
        {"id": task_id, "uid": user["id"]},
    ).mappings().first()
    if not row:
        raise HTTPException(404, "任務不存在或已處理")
    db.execute(
        text(
            "UPDATE applications SET status='aborted', status_reason='人工任務已取消',
updated_at=now() "
            "WHERE id=:aid"
        ),
        {"aid": row["application_id"]},
    )
    db.commit()
    return {"ok": True}
```

"""用戶認證：驗證前端傳來的 Supabase JWT 並注入當前用戶。

- 前端登錄/註冊走後端認證代理（app/api/auth.py），瀏覽器不直連 supabase.co
- 後端驗證 token：優先調 Supabase /auth/v1/user（權威），失敗退 JWKS 本地驗簽

- 進程內緩存驗證結果（5 分鐘），降低每個請求的網絡往返
- 首次訪問自動補建 profiles 行；ADMIN_EMAILS 中的郵箱自動提升 admin
"""
```
from __future__ import annotations

import logging
import time
from functools import lru_cache
from typing import Annotated

import httpx
import jwt
from fastapi import Depends, HTTPException, status
from fastapi.security import HTTPAuthorizationCredentials, HTTPBearer
from sqlalchemy import text
from sqlalchemy.orm import Session

from app.config import settings
from app.db import get_db

logger = logging.getLogger(__name__)

_bearer = HTTPBearer(auto_error=False)

# token -> (expire_ts, user_id, email)
_verify_cache: dict[str, tuple[float, str, str | None]] = {}
_CACHE_TTL = 300


@lru_cache(maxsize=1)
def _jwks() -> dict:
    url = f"{settings.supabase_url}/auth/v1/.well-known/jwks.json"
    try:
        resp = httpx.get(url, timeout=10)
        resp.raise_for_status()
        return resp.json()
    except httpx.HTTPError as exc:  # pragma: no cover
        logger.error("無法獲取 Supabase JWKS: %s", exc)
        raise RuntimeError("Supabase JWKS 不可用") from exc


def _verify_via_api(token: str) -> tuple[str, str | None] | None:
    """調 Supabase /auth/v1/user 驗證，返回 (user_id, email) 或 None。"""
    try:
        resp = httpx.get(
            f"{settings.supabase_url}/auth/v1/user",
            headers={
                "apikey": settings.supabase_anon_key,
                "Authorization": f"Bearer {token}",
            },
            timeout=10,
        )
        if resp.status_code == 200:
            data = resp.json()
            return data["id"], data.get("email")
    except httpx.HTTPError:
        pass
    return None


def _verify_local(token: str) -> tuple[str, str | None] | None:
    try:
        unverified = jwt.get_unverified_header(token)
        kid = unverified["kid"]
        key = next(k for k in _jwks()["keys"] if k["kid"] == kid)
```

```
        payload = jwt.decode(
            token,
            key=jwt.algorithms.RSAAlgorithm.from_jwk(key),
            algorithms=["RS256"],
            audience="authenticated",
            options={"verify_aud": True},
        )
        return payload.get("sub"), payload.get("email")
    except Exception:  # noqa: BLE001
        return None


def verify_token(token: str) -> tuple[str, str | None]:
    """驗證 token，返回 (user_id, email)。兩種方式都失敗 → 401。"""
    now = time.time()
    hit = _verify_cache.get(token)
    if hit and hit[0] > now:
        return hit[1], hit[2]
    result = _verify_via_api(token) or _verify_local(token)
    if not result:
        raise HTTPException(status.HTTP_401_UNAUTHORIZED, "登錄已失效，請重新登錄")
    try:
        exp = jwt.decode(token, options={"verify_signature": False}).get("exp")
        ttl = min(_CACHE_TTL, max(0, int(exp) - int(now))) if exp else _CACHE_TTL
    except Exception:  # noqa: BLE001
        ttl = _CACHE_TTL
    _verify_cache[token] = (now + ttl, result[0], result[1])
    return result


def get_current_user(
    credentials: Annotated[HTTPAuthorizationCredentials | None, Depends(_bearer)],
    db: Annotated[Session, Depends(get_db)],
) -> dict:
    """返回 {'id': uuid, 'email': str|None, 'phone': str|None, 'role': str}。"""
    if credentials is None:
        raise HTTPException(status.HTTP_401_UNAUTHORIZED, "缺少登錄憑證")
    user_id, email = verify_token(credentials.credentials)

    row = db.execute(
        text(
            """
            INSERT INTO profiles (id, email, phone)
            VALUES (:id, :email, :phone)
            ON CONFLICT (id) DO UPDATE
              SET email = COALESCE(profiles.email, EXCLUDED.email),
                  phone = COALESCE(profiles.phone, EXCLUDED.phone)
            RETURNING id::text, email, phone, role, full_name
            """
        ),
        {"id": user_id, "email": email, "phone": None},
    ).mappings().one()
    db.commit()
    result = dict(row)
    # 管理員自動提升：ADMIN_EMAILS 中的郵箱登錄即成為 admin
    email_lower = (result.get("email") or "").lower()
    if email_lower and email_lower in settings.admin_email_list and result.get("role") !=
"admin":
        db.execute(text("UPDATE profiles SET role='admin' WHERE id=:id"), {"id": user_id})
        db.commit()
        result["role"] = "admin"
    return result

"""信封加密（Envelope Encryption）——第三方平台帳密與登錄態的安全存儲。
```

```
方案（對應《01-架構設計》第 5 節）：
- 每用戶一把 DEK（Data Encryption Key，AES-256-GCM 隨機生成）
- DEK 用服務端 KEK（環境變量，不入庫）包裹後存 key_wraps 表
- 明文只存在於投遞 Worker 進程內存，永不落庫、不落日誌
"""
from __future__ import annotations

import base64
import os

from cryptography.hazmat.primitives.ciphers.aead import AESGCM

from app.config import settings

_NONCE_BYTES = 12


def _kek() -> bytes:
    raw = base64.b64decode(settings.kek_base64)
    if len(raw) != 32:
        raise RuntimeError("KEK_BASE64 必須為 32 字節的 base64（openssl rand -base64 32）")
    return raw


def _b64e(data: bytes) -> str:
    return base64.b64encode(data).decode("ascii")


def _b64d(data: str) -> bytes:
    return base64.b64decode(data.encode("ascii"))


def generate_dek() -> bytes:
    """生成用戶 DEK（32 字節）。"""
    return os.urandom(32)


def wrap_dek(dek: bytes) -> str:
    """用 KEK 包裹 DEK 後存庫。"""
    nonce = os.urandom(_NONCE_BYTES)
    ciphertext = AESGCM(_kek()).encrypt(nonce, dek, None)
    return _b64e(nonce + ciphertext)


def unwrap_dek(wrapped: str) -> bytes:
    """解包 DEK（僅在需要解密用戶數據時調用）。"""
    blob = _b64d(wrapped)
    nonce, ciphertext = blob[:_NONCE_BYTES], blob[_NONCE_BYTES:]
    return AESGCM(_kek()).decrypt(nonce, ciphertext, None)


def encrypt_field(dek: bytes, plaintext: str) -> str:
    """用用戶 DEK 加密單個字段，輸出 `nonce:ciphertext` base64。"""
    nonce = os.urandom(_NONCE_BYTES)
    ciphertext = AESGCM(dek).encrypt(nonce, plaintext.encode("utf-8"), None)
    return _b64e(nonce) + ":" + _b64e(ciphertext)


def decrypt_field(dek: bytes, stored: str) -> str:
    nonce_b64, ciphertext_b64 = stored.split(":", 1)
    return AESGCM(dek).decrypt(_b64d(nonce_b64), _b64d(ciphertext_b64), None).decode("utf-8")


def encrypt_json(dek: bytes, obj: dict) -> str:
    """加密 JSON 結構（如登錄態 Cookie 快照）。"""
```

```
    import json

    return encrypt_field(dek, json.dumps(obj, ensure_ascii=False))


def decrypt_json(dek: bytes, stored: str) -> dict:
    import json

    return json.loads(decrypt_field(dek, stored))


def get_dek_for_user(db, user_id: str) -> bytes:
    """取用戶 DEK；不存在則生成並包裹存庫（key_wraps）。"""
    from sqlalchemy import text

    row = db.execute(
        text(
            "SELECT wrapped_dek FROM key_wraps "
            "WHERE owner_type = 'user' AND owner_id = :uid"
        ),
        {"uid": user_id},
    ).mappings().first()
    if row:
        return unwrap_dek(row["wrapped_dek"])
    dek = generate_dek()
    db.execute(
        text(
            "INSERT INTO key_wraps (owner_type, owner_id, wrapped_dek) "
            "VALUES ('user', :uid, :wrapped)"
        ),
        {"uid": user_id, "wrapped": wrap_dek(dek)},
    )
    db.commit()
    return dek

"""審計日誌工具（不影響主流程，失敗僅告警）。"""
from __future__ import annotations

import logging

from sqlalchemy import text
from sqlalchemy.orm import Session

from app.core.jsonutil import j

logger = logging.getLogger(__name__)


def log_audit(
    db: Session,
    actor_user_id: str,
    action: str,
    entity: str | None = None,
    entity_id: str | None = None,
    meta: dict | None = None,
    ip: str | None = None,
) -> None:
    try:
        db.execute(
            text(
                """
                INSERT INTO audit_logs (actor_user_id, action, entity, entity_id, meta, ip)
                VALUES (:a, :action, :e, :eid, CAST(:meta AS jsonb), :ip)
                """
            ),
```

```
            {
                "a": actor_user_id,
                "action": action,
                "e": entity,
                "eid": entity_id,
                "meta": j(meta or {}),
                "ip": ip,
            },
        )
    except Exception:  # noqa: BLE001
        logger.exception("審計日誌寫入失敗")
```

```
"""R2 對象存儲客戶端（S3 兼容）。"""
from __future__ import annotations

import boto3
from botocore.config import Config

from app.config import settings

_client = None


def r2_client():
    global _client
    if _client is None:
        _client = boto3.client(
            "s3",
            endpoint_url=f"https://{settings.r2_account_id}.r2.cloudflarestorage.com",
            aws_access_key_id=settings.r2_access_key_id,
            aws_secret_access_key=settings.r2_secret_access_key,
            region_name="auto",
            config=Config(signature_version="s3v4"),
        )
    return _client


def upload_bytes(key: str, data: bytes, mime: str) -> None:
    r2_client().put_object(
        Bucket=settings.r2_bucket,
        Key=key,
        Body=data,
        ContentType=mime,
    )


def delete_object(key: str) -> None:
    r2_client().delete_object(Bucket=settings.r2_bucket, Key=key)


def object_url(key: str) -> str:
    if settings.r2_public_base_url:
        return f"{settings.r2_public_base_url.rstrip('/')}/{key}"
    return key


def presigned_url(key: str, expires: int = 3600) -> str:
    """私有桶的臨時訪問鏈接（無需公開桶）。"""
    return r2_client().generate_presigned_url(
        "get_object",
        Params={"Bucket": settings.r2_bucket, "Key": key},
        ExpiresIn=expires,
    )
```

```
"""貼合度匹配服務（FR-1f，GAP-12 雙層評分）。
```

```
第一層：硬性條件逐項核對（學歷/年限/簽證/語言/證書/地點），任一不滿足 → hard_failed。
第二層：LLM 綜合評分（50-100）+ 分項 + 可解釋理由。
結果 upsert 到 job_matches；status 由 auto_apply_mode 在投遞階段消費。
"""
from __future__ import annotations

from sqlalchemy import text
from sqlalchemy.orm import Session

from app.core.jsonutil import j
from app.core.llm import chat_json

_HARD_SYSTEM = """
你是一名招聘顧問。從 JD 中抽取硬性要求。
規則：
1. kind 取值：education(學歷/專業)、experience_years(工作年限)、visa(簽證/身份)、language(語言)、
certificate(證書)、location(工作地點/在場要求)、other。
2. must=true 表示"必須滿足"類要求（如"要求本科以上"、"需持綠卡"）；偏好類要求 must=false。
3. requirement 用一句話描述。
輸出 JSON（不要輸出其他內容）：
{"requirements": [{"field": "字段名", "kind": "...", "requirement": "描述", "must": true}]}
"""

_SCORE_SYSTEM = """
你是一名招聘顧問。綜合評估候選人與 JD 的貼合度，輸出 50-100 分。
規則：
1. 評分依據：技能重疊、經歷相關性、職級匹配、地點、薪資預期。
2. 若 JD 有硬性要求候選人不滿足，不要評分（此場景由系統先過濾，你不會收到）。
3. 給出三條簡潔可執行的理由（如"缺 XX 技能，可補充學習"）。
輸出 JSON（不要輸出其他內容）：
{"match_score": 50-100, "breakdown": {"skills_overlap": 0-100, "experience_fit": 0-100,
"level_fit": 0-100, "location_fit": 0-100, "salary_fit": 0-100}, "reasons": ["理由 1", "理由 2",
"理由 3"]}
"""

_SEMANTIC_SYSTEM = """
你是招聘篩選助手。候選人的資料是結構化字段 + 個人信息庫問答的合集。
規則檢查可能因為字段命名差異誤判，你需要用「聯想能力」判斷：
候選人資料中的任何一條信息（無論放在哪個字段）只要能**實質滿足**該要求，就判 pass=true。
例如：要求「熟練使用中英文作為工作語言」，信息庫寫「英文：專業（可作為工作語言）」→ pass=true。
要求「碩士學歷」而教育經歷最高為本科 → pass=false（不要放水）。
輸出 JSON（不要輸出其他內容）：
{"results": [{"index": 0, "pass": true, "note": "一句話原因"}]}
"""


def extract_hard_requirements(jd_text: str) -> list[dict]:
    out = chat_json(_HARD_SYSTEM, f"JD: \n{jd_text[:8000]}")
    return [r for r in (out.get("requirements") or []) if r.get("must")]


def _check_against(profile: dict, req: dict) -> bool:
    """簡化的規則核對 + 兜底。返回是否通過。"""
    kind = req.get("kind")
    requirement = (req.get("requirement") or "").lower()
    hard = profile.get("hard_facts") or {}

    if kind == "experience_years":
        years = hard.get("total_years")
        if years is None:
            return True  # 無法核對不攔截，交由 LLM 評分
        import re

        m = re.search(r"(\d+)", requirement)
```

```
        return years >= int(m.group(1)) if m else True
    if kind == "visa":
        return bool(hard.get("visa_status"))  # 有簽證信息即放行，細粒度在 LLM 評分
    if kind == "language":
        langs = " ".join(profile.get("languages") or [])
        return bool(langs)  # 有語言信息即放行，具體匹配度在 LLM 評分
    if kind == "certificate":
        certs = " ".join(profile.get("certificates") or [])
        return bool(certs)
    if kind == "location":
        locs = " ".join(hard.get("preferred_locations") or [])
        return bool(locs)
    return True


def _semantic_recheck(db: Session, user_id: str, profile: dict, requirements: list[dict]) ->
list[dict]:
    """聯想能力兜底：規則判定失敗的要求，用 LLM 帶候選人全量資料語義確認。"""
    info = db.execute(
        text("SELECT key, value FROM user_info_fields WHERE user_id=:uid"),
        {"uid": user_id},
    ).mappings().all()
    candidate = {
        "硬性事實": profile.get("hard_facts") or {},
        "語言能力": profile.get("languages") or [],
        "技能": [s.get("name") if isinstance(s, dict) else s for s in (profile.get("skills") or
[])],
        "教育": profile.get("education") or [],
        "經歷": profile.get("experiences") or [],
        "個人信息庫": {r["key"]: r["value"] for r in info},
    }
    out = chat_json(
        _SEMANTIC_SYSTEM,
        f"候選人資料：{j(candidate)}\n待確認要求：{j(requirements)}",
    )
    results = out.get("results") or []
    by_index = {int(r.get("index", -1)): r for r in results if isinstance(r, dict)}
    return [
        f for i, f in enumerate(requirements)
        if not (by_index.get(i) or {}).get("pass", False)
    ]


def score_posting(db: Session, user_id: str, posting_id: str) -> dict:
    """對單個崗位執行雙層評分並落庫。返回 match 記錄。"""
    profile = db.execute(
        text(
            "SELECT * FROM candidate_profiles WHERE user_id=:uid AND is_current=true "
            "ORDER BY generated_at DESC LIMIT 1"
        ),
        {"uid": user_id},
    ).mappings().first()
    posting = db.execute(
        text("SELECT * FROM job_postings WHERE id=:pid"),
        {"pid": posting_id},
    ).mappings().first()
    if not profile:
        raise ValueError("請先生成用戶畫像")
    if not posting:
        raise ValueError("崗位不存在")

    # 統一信息源：元信息表（含語言能力等問答）合併進畫像，兩邊永不失同步
    info = db.execute(
        text("SELECT key, value FROM user_info_fields WHERE user_id=:uid"),
        {"uid": user_id},
```

```
    ).mappings().all()
    info_map = {r["key"]: r["value"] for r in info}
    profile = dict(profile)
    hf = dict(profile.get("hard_facts") or {})
    for k, v in info_map.items():
        if k in ("visa_status", "preferred_locations", "salary_expectation",
                 "availability", "total_years", "candidate_name"):
            hf[k] = v
    # total_years 統一為數字（信息庫可能存字符串）
    try:
        hf["total_years"] = float(hf.get("total_years") or 0)
    except (TypeError, ValueError):
        hf["total_years"] = 0.0
    profile["hard_facts"] = hf
    # 語言能力：info_fields 的 languages/語言能力 key 轉為列表供硬性條件檢查
    lang_val = info_map.get("languages") or info_map.get("語言能力") or info_map.get("语言能力")
    if lang_val:
        profile["languages"] = [s.strip() for s in str(lang_val).replace("、", ",").split(",") if
s.strip()]
    elif hf.get("languages"):
        profile["languages"] = hf["languages"] if isinstance(hf["languages"], list) else
[str(hf["languages"])]

    hard_reqs = extract_hard_requirements(posting["description_text"] or posting["title"])
    # location 類要求不硬攔截（交給地區加權處理：命中加分、未命中降分）
    failures = [
        r for r in hard_reqs
        if r.get("kind") != "location" and not _check_against(dict(profile), r)
    ]
    # 聯想能力兜底：規則失敗的要求交由 LLM 語義二次確認（字段命名差異不再誤殺）
    if failures:
        try:
            failures = _semantic_recheck(db, user_id, dict(profile), failures)
        except Exception:  # noqa: BLE001
            logger.exception("硬性要求語義確認失敗，按規則結果處理 user=%s", user_id)

    # 校招/社招偏好與職級適配（投遞規則 R2 修訂版）：
    # - 用戶偏好只投社招 → 校招崗位硬性過濾
    # - 用戶允許投校招 → 有工作經歷者不攔截，但降權（職級不適配）
    campus_adjust = 0
    st = db.execute(
        text("SELECT campus_preference FROM user_settings WHERE user_id=:u"),
        {"u": user_id},
    ).mappings().first()
    preference = (st["campus_preference"] if st else "both") or "both"
    is_campus = bool(posting.get("is_campus")) or _is_campus_posting(posting)
    years = (dict(profile).get("hard_facts") or {}).get("total_years") or 0
    if is_campus and preference == "society":
        failures.append({
            "field": "campus_preference",
            "kind": "other",
            "requirement": "用戶設置為只投社招，校招崗位已過濾",
            "passed": False,
        })
    elif is_campus and years >= 1:
        campus_adjust = -15  # 校招職級不適配（有工作經歷）降權
    elif (not is_campus) and preference == "campus":
        failures.append({
            "field": "campus_preference",
            "kind": "other",
            "requirement": "用戶設置為只投校招，社招崗位已過濾",
            "passed": False,
        })

    if failures:
```

```
        db.execute(
            text(
                """
                INSERT INTO job_matches (user_id, posting_id, profile_version_id, hard_failures,
status)
                VALUES (:uid, :pid, :pv, CAST(:hf AS jsonb), 'hard_failed')
                ON CONFLICT (user_id, posting_id) DO UPDATE
                  SET hard_failures=EXCLUDED.hard_failures, status='hard_failed',
updated_at=now()
                """
            ),
            {"uid": user_id, "pid": posting_id, "pv": profile["id"], "hf": j(failures)},
        )
        db.commit()
        return {"match_score": None, "hard_failures": failures, "status": "hard_failed"}

    jd = (posting["description_text"] or "")[:8000]
    prompt = (
        f"候選人概覽: {profile['summary']}\n"
        f"技能: {profile['skills']}\n"
        f"經歷: {profile['experiences']}\n"
        f"硬性事實: {profile['hard_facts']}\n\n"
        f"JD: {jd}"
    )

    # ── 崗位方向: 逐方向分別評分（投遞仍一崗一投；崗位級分數取最高方向）──
    raw = posting.get("raw") or {}
    if isinstance(raw, str):
        try:
            raw = json.loads(raw)
        except ValueError:
            raw = {}
    directions = [d for d in (raw.get("directions") or []) if d]
    direction_scores: list[dict] = []
    if len(directions) > 1:
        for d in directions:
            try:
                s = chat_json(_SCORE_SYSTEM, prompt + f"\n（本崗位方向: {d}。請只針對該方向評分。）")
                direction_scores.append({
                    "direction": d,
                    "match_score": min(max(int(s["match_score"]), 50), 100),
                    "reasons": s.get("reasons") or [],
                })
            except Exception:  # noqa: BLE001
                logger.warning("方向評分失敗 direction=%s posting=%s", d, posting_id)
        if direction_scores:
            best = max(direction_scores, key=lambda x: x["match_score"])
            scored = {"match_score": best["match_score"], "breakdown": {},
                      "reasons": best["reasons"] + [f"（按最匹配方向「{best['direction']}」計）"]}
        else:
            scored = chat_json(_SCORE_SYSTEM, prompt)
    else:
        scored = chat_json(_SCORE_SYSTEM, prompt)

    # ── 貼合度 = 純個人條件匹配（LLM），不做任何偏好加減 ──
    fit_score = min(max(int(scored["match_score"]), 50), 100)

    # ── 投遞優先級 = 貼合度 + 偏好加權（地區/校招職級/總部分支），只用於排序與投遞選擇 ──
    loc_adj = _location_adjust(db, user_id, posting["location"])
    org_adj = _org_adjust(db, user_id, posting.get("org_level") or "unknown")
    priority = min(max(fit_score + loc_adj + campus_adjust + org_adj, 50), 100)
    breakdown = scored.get("breakdown") or {}
    breakdown.update({
        "fit_base": fit_score,
        "location_adjust": loc_adj,
```

```
        "campus_adjust": campus_adjust,
        "org_adjust": org_adj,
    })

    db.execute(
        text(
            """
            INSERT INTO job_matches
              (user_id, posting_id, profile_version_id, match_score, priority_score,
               score_breakdown, reasons, direction_scores, status)
            VALUES (:uid, :pid, :pv, :score, :prio, CAST(:bd AS jsonb), CAST(:reasons AS jsonb),
                    CAST(:ds AS jsonb), 'new')
            ON CONFLICT (user_id, posting_id) DO UPDATE
              SET match_score=EXCLUDED.match_score, priority_score=EXCLUDED.priority_score,
                  score_breakdown=EXCLUDED.score_breakdown,
                  reasons=EXCLUDED.reasons, direction_scores=EXCLUDED.direction_scores,
                  hard_failures=NULL,
                  status=CASE WHEN job_matches.status IN ('applied','applying') THEN
job_matches.status ELSE 'new' END,
                  updated_at=now()
            """
        ),
        {
            "uid": user_id,
            "pid": posting_id,
            "pv": profile["id"],
            "score": fit_score,
            "prio": priority,
            "bd": j(breakdown),
            "reasons": j(scored.get("reasons") or []),
            "ds": j(direction_scores or []),
        },
    )
    db.commit()
    return {"match_score": fit_score, "priority_score": priority, "hard_failures": [],
            "status": "new", "location_adjust": loc_adj,
            "direction_scores": direction_scores}


def _org_adjust(db: Session, user_id: str, org_level: str) -> int:
    """總部/分支偏好加權：偏好總部時分支 -8；偏好分支時總部 -5；一視同仁 0。"""
    st = db.execute(
        text("SELECT org_preference FROM user_settings WHERE user_id=:u"),
        {"u": user_id},
    ).mappings().first()
    pref = (st["org_preference"] if st else "both") or "both"
    if pref == "headquarter" and org_level == "branch":
        return -8
    if pref == "branch" and org_level == "headquarter":
        return -5
    return 0


def _is_campus_posting(posting: dict) -> bool:
    """判斷是否校招/應屆類崗位（標題/JD/採集元數據）。"""
    raw = posting.get("raw") or {}
    if isinstance(raw, str):  # 防禦：raw 偶爾以字符串返回
        try:
            import json as _json

            raw = _json.loads(raw)
        except ValueError:
            raw = {}
    text = " ".join([
        posting.get("title") or "",
```

```
        posting.get("description_text") or "",
        str(raw.get("tab") or ""),
        str(raw.get("recruit_type") or ""),
    ]).lower()
    hints = ("校招", "校园", "应届", "應屆", "管培生", "培训生", "培訓生", "秋招", "春招", "2027届",
"2027屆")
    return any(h in text for h in hints)


def _location_adjust(db: Session, user_id: str, posting_location: str | None) -> int:
    """期望城市命中 +location_weight；未命中且配置了期望城市 → -min(weight,10)。"""
    st = db.execute(
        text("SELECT preferred_locations, location_weight FROM user_settings WHERE user_id=:u"),
        {"u": user_id},
    ).mappings().first()
    preferred = (st["preferred_locations"] if st else None) or []
    weight = (st["location_weight"] if st else None) or 10
    if not preferred:
        return 0
    loc = (posting_location or "").lower()
    if any(p.lower() in loc for p in preferred):
        return weight
    return -min(weight, 10)

"""投遞引擎（FR-3b/3c）：狀態機、預檢、文檔準備、適配器調度、重試、人工接管、結果通知。

狀態流轉（對應《01-架構設計》4.3）：
queued → in_progress → submitted / needs_input(→answered→queued) / failed_retryable / aborted
"""
from __future__ import annotations

import logging
import os
import tempfile
from datetime import datetime, timedelta, timezone

from sqlalchemy import text
from sqlalchemy.orm import Session

from app.apply import base as apply_base
from app.config import settings as app_settings
from app.core.audit import log_audit
from app.core.jsonutil import j
from app.services.form_mapping import build_candidate_payload, missing_fields
from app.api.info_fields import save_answer
from app.services.notifier import notify_with_fallback
from app.services.optimize import (
    attach_cover_letter,
    generate_cover_letter,
    optimize_resume_for_jd,
)
from app.services.parsing import download_bytes

logger = logging.getLogger(__name__)

MAX_RETRIES = 3
RETRY_BASE_MINUTES = 30
QUEUE_LIMIT = 50  # 每輪 process_queue 處理上限（免費層資源控制）

# 人工回填允許覆蓋的字段白名單（防止注入任意鍵）
ANSWER_WHITELIST = {
    "full_name", "email", "phone", "location", "visa_status", "availability",
    "salary_expectation", "linkedin_url", "portfolio_url", "sms_code",
}
```

```
# ── 1. 隊列生成（FR-1g 三態策略）
# ──────────────────────────────────────────────────────────────

def create_applications_from_matches(db: Session, user_id: str) -> dict:
    st = db.execute(
        text(
            "SELECT auto_apply_mode, match_threshold, campus_preference "
            "FROM user_settings WHERE user_id=:uid"
        ),
        {"uid": user_id},
    ).mappings().first()
    mode = st["auto_apply_mode"] if st else "threshold"
    if mode == "off":
        return {"mode": "off", "created": 0}
    threshold = st["match_threshold"] if st else 70
    pref = (st["campus_preference"] if st else "both") or "both"
    where = "m.status IN ('new','pushed') AND m.match_score IS NOT NULL"
    params: dict = {"uid": user_id}
    if mode == "threshold":
        where += " AND m.match_score >= :th"
        params["th"] = threshold
    campus_where = ""
    if pref == "society":
        campus_where = "AND p.is_campus = false"
    elif pref == "campus":
        campus_where = "AND p.is_campus = true"
    result = db.execute(
        text(
            f"""
            INSERT INTO applications (user_id, posting_id, match_id, status)
            SELECT m.user_id, m.posting_id, m.id, 'queued'
            FROM job_matches m JOIN job_postings p ON p.id = m.posting_id
            WHERE m.user_id = :uid AND {where} AND p.is_active = true
              {campus_where}
              -- 公司級一崗一投：同一公司已有未終止投遞時跳過其餘崗位
              AND NOT EXISTS (
                SELECT 1 FROM applications a2
                JOIN job_postings p2 ON p2.id = a2.posting_id
                WHERE a2.user_id = :uid AND p2.company_name = p.company_name
                  AND a2.status NOT IN ('aborted', 'duplicate_skip', 'not_supported')
              )
            ORDER BY m.priority_score DESC NULLS LAST, m.match_score DESC NULLS LAST
            ON CONFLICT (user_id, posting_id) DO NOTHING
            """
        ),
        params,
    )
    db.execute(
        text(
            f"""
            UPDATE job_matches SET status='applying', updated_at=now()
            WHERE user_id = :uid AND status IN ('new','pushed')
              AND match_score IS NOT NULL
              {("AND match_score >= :th") if mode == 'threshold' else ''}
            """
        ),
        params,
    )
    db.commit()
    return {"mode": mode, "created": result.rowcount}
```

```
# ── 2. 預檢 ──────────────────────────────────────────────────────────────────


def _preflight(payload: dict) -> dict | None:
    """返回阻塞原因（manual request 描述），無阻塞返回 None。"""
    miss = missing_fields(payload)
    if miss["critical"]:
        return {
            "task_type": "missing_field",
            "question": f"投遞前缺關鍵信息：{', '.join(miss['critical'])}。請補全後重試。",
            "options": {f: {"label": f, "type": "text"} for f in miss["critical"]},
        }
    if not payload.get("resume_file"):
        return {
            "task_type": "missing_file",
            "question": "尚未上傳可用簡歷（需上傳並解析成功）。請先在「我的文件」上傳簡歷。",
            "options": {"confirmed": {"label": "我已上傳簡歷", "type": "checkbox"}},
        }
    return None


# ── 3. 上下文準備（簡歷版本選擇 + AI 微調 + CL + 本地文件）──────────────────

# 判斷需要英文簡歷的地點/地區關鍵詞（中國大陸以外）
_EN_HINTS = (
    "hong kong", "香港", "singapore", "london", "new york", "san francisco",
    "tokyo", "seoul", "sydney", "toronto", "vancouver", "remote", "海外",
    "united states", "united kingdom", "canada", "australia",
)


def _pick_resume_lang(app: dict) -> str:
    """根據崗位地點/公司選擇簡歷語言：海外/香港 → en，否則 zh。"""
    location = (app.get("location") or "").lower()
    text = f"{location} {(app.get('description_text') or '')[:300]}".lower()
    return "en" if any(h in text for h in _EN_HINTS) else "zh"


def _prepare_context(db: Session, app: dict, payload: dict, settings_row: dict) ->
apply_base.ApplyContext:
    want_lang = _pick_resume_lang(app)
    version = db.execute(
        text(
            "SELECT id, content_md FROM resume_versions WHERE user_id=:uid "
            "AND content_md IS NOT NULL AND COALESCE(lang,'zh')=:lang "
            "ORDER BY version_no DESC LIMIT 1"
        ),
        {"uid": app["user_id"], "lang": want_lang},
    ).mappings().first()
    if not version:
        # 沒有對應語言的簡歷時回退到最新任意版本
        version = db.execute(
            text(
                "SELECT id, content_md FROM resume_versions WHERE user_id=:uid "
                "AND content_md IS NOT NULL ORDER BY version_no DESC LIMIT 1"
            ),
            {"uid": app["user_id"]},
        ).mappings().first()
    if not version:
        raise RuntimeError("沒有可用的簡歷文本（請先解析簡歷）")
    version_id, resume_text = str(version["id"]), version["content_md"]

    jd = app.get("description_text")
    if settings_row.get("ai_optimize_resume") and jd:
```

```
        new_vid = optimize_resume_for_jd(db, app["user_id"], version_id, jd)
        if new_vid:
            new_row = db.execute(
                text("SELECT content_md FROM resume_versions WHERE id=:id"),
                {"id": new_vid},
            ).mappings().first()
            version_id, resume_text = new_vid, new_row["content_md"]
            db.execute(
                text("UPDATE applications SET resume_version_id=:rv, updated_at=now() WHERE
id=:id"),
                {"rv": version_id, "id": app["id"]},
            )
            db.commit()

    cover_letter = None
    if settings_row.get("ai_optimize_cover_letter") and jd:
        cover_letter = generate_cover_letter(db, app["user_id"], resume_text, jd)
        if cover_letter:
            attach_cover_letter(db, version_id, cover_letter)

    local_path = _download_resume(payload)

    account = _load_account(db, app)

    return apply_base.ApplyContext(
        user_id=app["user_id"],
        application_id=app["id"],
        posting=app,
        payload=payload,
        resume_text=resume_text,
        cover_letter=cover_letter,
        resume_local_path=local_path,
        settings=settings_row,
        account=account,
    )


def _load_account(db: Session, app: dict) -> dict | None:
    """解密平台帳號（按投遞來源匹配：gtht 等官網登錄型投遞）。

  匹配規則：來源為 gtht 時 → 從「公司官網/gtht」帳號中按網址或標籤
  匹配國泰海通；否則按 platform=來源 → 回退第一個 official 帳號。
  返回 {id, login, password}。
    """
    from app.core.security import decrypt_field, decrypt_json, get_dek_for_user

    dek = None

    def _decrypt(row) -> dict | None:
        nonlocal dek
        try:
            if dek is None:
                dek = get_dek_for_user(db, app["user_id"])
            return {
                "id": str(row["id"]),
                "login": decrypt_field(dek, row["login_identifier_enc"]),
                "password": decrypt_field(dek, row["password_enc"]) or "",
            }
        except Exception as exc:  # noqa: BLE001
            logger.warning("平台帳號解密失敗 user=%s: %s", app["user_id"], exc)
            return None

    if app["source"] == "gtht":
        rows = db.execute(
            text(
```

```
                "SELECT id, platform, label, login_identifier_enc, password_enc, meta_enc "
                "FROM platform_accounts "
                "WHERE user_id=:uid AND status != 'locked' "
                "AND (platform IN ('gtht','official') "
                "     OR label ILIKE '%國泰%' OR label ILIKE '%国泰%' OR label ILIKE '%gtht%') "
                "ORDER BY created_at"
            ),
            {"uid": app["user_id"]},
        ).mappings().all()
        for row in rows:
            meta = {}
            if row["meta_enc"]:
                try:
                    if dek is None:
                        dek = get_dek_for_user(db, app["user_id"])
                    meta = decrypt_json(dek, row["meta_enc"])
                except Exception:  # noqa: BLE001
                    meta = {}
            website = str(meta.get("website") or "").lower()
            label = (row["label"] or "").lower()
            if (row["platform"] == "gtht" or "gtht" in website
                    or "國泰" in label or "国泰" in label or "gtht" in label):
                return _decrypt(row)

    if app["source"] == "citics":
        # 中信帳號存 platform='official'，按網址/標籤匹配（platform_name 枚舉無 citics）
        rows = db.execute(
            text(
                "SELECT id, platform, label, login_identifier_enc, password_enc, meta_enc "
                "FROM platform_accounts "
                "WHERE user_id=:uid AND status != 'locked' "
                "AND (label ILIKE '%中信%' OR label ILIKE '%citics%') "
                "ORDER BY created_at"
            ),
            {"uid": app["user_id"]},
        ).mappings().all()
        for row in rows:
            return _decrypt(row)

    # platform 需在枚舉內，否則枚舉非法值會中止事務
    valid_platforms = ("workday", "greenhouse", "lever", "successfactors", "icims",
                       "official", "liepin", "boss", "lagou", "zhilian", "job51",
                       "linkedin", "indeed", "gtht", "other")
    for platform in (app["source"], "official"):
        if platform not in valid_platforms:
            continue
        row = db.execute(
            text(
                "SELECT id, login_identifier_enc, password_enc FROM platform_accounts "
                "WHERE user_id=:uid AND platform=:p AND status != 'locked' "
                "ORDER BY created_at LIMIT 1"
            ),
            {"uid": app["user_id"], "p": platform},
        ).mappings().first()
        if row:
            return _decrypt(row)
    return None


def _download_resume(payload: dict) -> str | None:
    rf = payload.get("resume_file")
    if not rf:
        return None
    data = download_bytes(rf["storage_path"])
    ext = os.path.splitext(rf["name"])[1] or ".pdf"
```

```
    fd, path = tempfile.mkstemp(suffix=ext, prefix="jobfly_resume_")
    with os.fdopen(fd, "wb") as f:
        f.write(data)
    return path


# ── 4. 主流程 ──────────────────────────────────────────────────────────────

def run_application(db: Session, application_id: str, dry_run: bool = False, force: bool =
False) -> dict:
    app = _load_application(db, application_id)
    if not app:
        return {"error": "application not found"}
    if app["status"] in ("submitted", "aborted", "not_supported", "duplicate_skip"):
        return {"skipped": app["status"]}
    if app["status"] == "in_progress":
        return {"skipped": "in_progress"}
    if app["status"] == "needs_input" and not force:
        return {"skipped": "needs_input"}

    attempt_id = db.execute(
        text(
            "INSERT INTO application_attempts (user_id, application_id, attempt_no, result) "
            "VALUES (:uid, :aid, :n, 'failed') RETURNING id"
        ),
        {"uid": app["user_id"], "aid": application_id, "n": app["attempts_count"] + 1},
    ).mappings().one()["id"]
    db.execute(
        text(
            "UPDATE applications SET status='in_progress', current_step='prepare', "
            "attempts_count=attempts_count+1, updated_at=now() WHERE id=:id"
        ),
        {"id": application_id},
    )
    db.commit()

    outcome: dict = {}
    local_path: str | None = None
    try:
        if not app.get("apply_url"):
            _finish(db, app, application_id, "not_supported", "該崗位無申請入口", attempt_id,
"aborted")
            outcome = {"status": "not_supported"}
        else:
            payload = build_candidate_payload(db, app["user_id"])
            _merge_answers(db, application_id, payload)
            blocker = _preflight(payload)
            if blocker:
                outcome = _block(db, app, application_id, attempt_id, blocker)
            else:
                settings_row = db.execute(
                    text("SELECT * FROM user_settings WHERE user_id=:uid"),
                    {"uid": app["user_id"]},
                ).mappings().first()
                ctx = _prepare_context(db, app, payload, dict(settings_row) if settings_row else
{})
                local_path = ctx.resume_local_path
                ctx.dry_run = dry_run
                adapter = apply_base.get_apply_adapter(app["source"])
                db.execute(
                    text("UPDATE applications SET current_step='applying', updated_at=now()
WHERE id=:id"),
                    {"id": application_id},
```

```
                )
                db.commit()
                result = adapter.apply(ctx)
                outcome = _settle(db, app, application_id, attempt_id, result)
    except Exception as exc:  # noqa: BLE001
        logger.exception("投遞執行異常 app=%s", application_id)
        try:
            db.rollback()  # 若事務已中止，先回滾再收尾（防 InFailedSqlTransaction 連鎖）
        except Exception:  # noqa: BLE001
            pass
        outcome = _finish(
            db, app, application_id,
            "failed_retryable" if app["attempts_count"] < MAX_RETRIES else "aborted",
            f"內部錯誤: {exc}", attempt_id, "failed", error_code="internal_error",
        )
    finally:
        db.execute(
            text("UPDATE application_attempts SET finished_at=now() WHERE id=:id AND finished_at
IS NULL"),
            {"id": attempt_id},
        )
        if local_path:
            try:
                os.unlink(local_path)
            except OSError:  # pragma: no cover
                pass
        db.commit()
    return outcome


def _load_application(db: Session, application_id: str) -> dict | None:
    row = db.execute(
        text(
            """
            SELECT a.*, p.company_name, p.title, p.location, p.source, p.apply_url,
                   p.description_text, p.external_id AS posting_external_id
            FROM applications a JOIN job_postings p ON p.id = a.posting_id
            WHERE a.id = :id
            """
        ),
        {"id": application_id},
    ).mappings().first()
    if not row:
        return None
    d = dict(row)
    d["id"] = str(d["id"])
    d["user_id"] = str(d["user_id"])
    return d


def _block(db: Session, app: dict, application_id: str, attempt_id: str, blocker: dict) -> dict:
    task_id = _create_manual_task(db, app, application_id, blocker)
    _finish(db, app, application_id, "needs_input", blocker["question"], attempt_id,
            "manual_input_needed", error_code=blocker["task_type"])
    _notify_manual(db, app, blocker["question"], task_id)
    return {"status": "needs_input", "task_id": task_id}


def _settle(db: Session, app: dict, application_id: str, attempt_id: str,
            result: apply_base.ApplyResult) -> dict:
    if result.success:
        db.execute(
            text("UPDATE job_matches SET status='applied', updated_at=now() WHERE id=:mid"),
            {"mid": app["match_id"]},
        )
```

```
        _finish(db, app, application_id, "submitted",
                f"投遞成功 ref={result.external_ref or ''}", attempt_id, "success")
        _notify_result(db, app, success=True, detail=result.external_ref or "")
        return {"status": "submitted", "external_ref": result.external_ref}
    if result.manual:
        blocker = {
            "task_type": result.manual.task_type,
            "question": result.manual.question,
            "options": result.manual.options,
            "screenshot_path": result.manual.screenshot_path,
        }
        task_id = _create_manual_task(db, app, application_id, blocker)
        _finish(db, app, application_id, "needs_input", result.manual.question, attempt_id,
                "manual_input_needed", error_code=result.error_code or result.manual.task_type)
        _notify_manual(db, app, result.manual.question, task_id)
        return {"status": "needs_input", "task_id": task_id}
    # 可重試失敗
    if app["attempts_count"] < MAX_RETRIES:
        retry_at = datetime.now(timezone.utc) + timedelta(minutes=RETRY_BASE_MINUTES *
app["attempts_count"])
        db.execute(
            text("UPDATE applications SET next_retry_at=:t WHERE id=:id"),
            {"t": retry_at, "id": application_id},
        )
        _finish(db, app, application_id, "failed_retryable",
                result.error_detail or result.error_code or "投遞失敗", attempt_id, "failed",
                error_code=result.error_code)
        return {"status": "failed_retryable", "error": result.error_detail}
    _finish(db, app, application_id, "aborted",
            result.error_detail or "多次重試仍失敗", attempt_id, "aborted",
error_code=result.error_code)
    _notify_result(db, app, success=False, detail=result.error_detail or result.error_code or
"")
    return {"status": "aborted", "error": result.error_detail}


def _merge_answers(db: Session, application_id: str, payload: dict) -> None:
    """把最近一次人工回填合併進投遞上下文（FR-3c 直接框 → 繼續投遞）。"""
    task = db.execute(
        text(
            "SELECT answer FROM manual_tasks WHERE application_id=:aid AND status='answered' "
            "ORDER BY answered_at DESC LIMIT 1"
        ),
        {"aid": application_id},
    ).mappings().first()
    if not task or not task["answer"]:
        return
    for key, value in task["answer"].items():
        if key == "confirmed" or value is None:
            continue
        if key == "values" and isinstance(value, str):
            for line in value.splitlines():
                if "=" in line:
                    k, v = line.split("=", 1)
                    k = k.strip()
                    if k in payload and v.strip():
                        payload[k] = v.strip()
        elif isinstance(value, str) and value.strip() and key in ANSWER_WHITELIST:
            payload[key] = value.strip()


def _finish(db: Session, app: dict, application_id: str, status: str, reason: str,
            attempt_id: str, attempt_result: str, error_code: str | None = None) -> dict:
    db.execute(
        text(
```

```
            "UPDATE applications SET status=CAST(:st AS application_status), status_reason=:r, "
            "current_step=NULL, "
            "submitted_at=CASE WHEN :st='submitted' THEN now() ELSE submitted_at END, "
            "updated_at=now() WHERE id=:id"
        ),
        {"st": status, "r": reason[:2000], "id": application_id},
    )
    db.execute(
        text(
            "UPDATE application_attempts SET result=CAST(:r AS attempt_result), error_code=:ec "
            "WHERE id=:id"
        ),
        {"r": attempt_result, "ec": error_code, "id": attempt_id},
    )
    log_audit(db, app["user_id"], "application.status", "applications", application_id,
              {"status": status, "reason": reason[:300]})
    db.commit()
    return {"status": status}


def _create_manual_task(db: Session, app: dict, application_id: str, blocker: dict) -> str:
    task = db.execute(
        text(
            """
            INSERT INTO manual_tasks
              (user_id, application_id, task_type, question, options, screenshot_path,
               status, expires_at)
            VALUES (:uid, :aid, CAST(:tt AS task_type), :q, CAST(:opt AS jsonb), :shot, 'open',
now() + interval '48 hours')
            RETURNING id
            """
        ),
        {
            "uid": app["user_id"], "aid": application_id, "tt": blocker["task_type"],
            "q": blocker["question"], "opt": j(blocker.get("options")),
            "shot": blocker.get("screenshot_path"),
        },
    ).mappings().one()
    db.commit()
    return str(task["id"])


# ── 5. 隊列處理（調度器/手動觸發） ──────────────────────────────────────────


def process_queue(db: Session, user_id: str | None = None, limit: int = QUEUE_LIMIT) -> dict:
    where = "status='queued' OR (status='failed_retryable' AND next_retry_at <= now())"
    params: dict = {"lim": limit}
    if user_id:
        where = f"({where}) AND user_id = :uid"
        params["uid"] = user_id
    rows = db.execute(
        text(
            f"SELECT id FROM applications WHERE {where} ORDER BY created_at LIMIT :lim"
        ),
        params,
    ).mappings().all()
    done = {"processed": 0, "submitted": 0, "needs_input": 0, "failed": 0, "skipped": 0}
    for r in rows:
        app_id = str(r["id"])
        if not _rate_ok(db, app_id):
            done["skipped"] += 1
            continue
        out = run_application(db, app_id)
        if out.get("skipped"):
```

```
            done["skipped"] += 1
            continue
        done["processed"] += 1
        if out.get("status") == "submitted":
            done["submitted"] += 1
        elif out.get("status") == "needs_input":
            done["needs_input"] += 1
        else:
            done["failed"] += 1
    return done


def _rate_ok(db: Session, application_id: str) -> bool:
    """每用戶投遞最小間隔（apply_rate_limit_min），防止風控與資源耗盡。"""
    row = db.execute(
        text(
            """
            SELECT s.apply_rate_limit_min AS rate,
                   (SELECT MAX(at2.started_at) FROM application_attempts at2
                    JOIN applications a2 ON a2.id = at2.application_id
                    WHERE a2.user_id = a.user_id) AS last
            FROM applications a JOIN user_settings s ON s.user_id = a.user_id
            WHERE a.id = :id
            """
        ),
        {"id": application_id},
    ).mappings().first()
    if not row:
        return True
    rate = row["rate"] or 5
    if not row["last"]:
        return True
    elapsed = (datetime.now(timezone.utc) - row["last"]).total_seconds() / 60
    if elapsed < rate:
        db.execute(
            text(
                "UPDATE applications SET next_retry_at=:t, updated_at=now() WHERE id=:id"
            ),
            {"t": row["last"] + timedelta(minutes=rate), "id": application_id},
        )
        db.commit()
        return False
    return True


# ── 6. 人工接管回填 ──────────────────────────────────────────────


def answer_and_resume(db: Session, task_id: str, user_id: str, answer: dict | None) -> dict:
    task = db.execute(
        text("SELECT * FROM manual_tasks WHERE id=:id AND user_id=:uid"),
        {"id": task_id, "uid": user_id},
    ).mappings().first()
    if not task:
        return {"error": "task not found"}
    if task["status"] != "open":
        return {"skipped": task["status"]}
    db.execute(
        text(
            "UPDATE manual_tasks SET status='answered', answered_at=now(), answer=CAST(:a AS
jsonb) "
            "WHERE id=:id"
        ),
        {"id": task_id, "a": j(answer or {})},
    )
```

```
    # 回填答案自動存庫（元信息表），下次同類字段免問
    for key, value in (answer or {}).items():
        if key in ("confirmed",) or value in (None, ""):
            continue
        if isinstance(value, str) and value.strip():
            try:
                save_answer(db, user_id, key, value.strip())
            except Exception:  # noqa: BLE001
                logger.warning("元信息存庫失敗 %s=%s", key, value[:50])
    db.commit()
    # 適配器專屬人工回填（如中信：用戶官網自行提交後確認 → 直接標記已投遞）
    if task["task_type"] == "citics_manual":
        try:
            from app.apply.citics_apply import merge_manual_answer

            app_row = _load_application(db, str(task["application_id"]))
            payload = build_candidate_payload(db, user_id)
            _merge_answers(db, str(task["application_id"]), payload)
            ctx = _prepare_context(db, app_row, payload, {})
            merged = merge_manual_answer(ctx, answer or {})
            if merged.get("submitted"):
                db.execute(
                    text("SELECT id FROM application_attempts WHERE application_id=:aid ORDER BY
created_at DESC LIMIT 1"),
                    {"aid": task["application_id"]},
                )
                attempt = db.execute(
                    text("SELECT id FROM application_attempts WHERE application_id=:aid ORDER BY
created_at DESC LIMIT 1"),
                    {"aid": task["application_id"]},
                ).mappings().first()
                _finish(db, app_row, str(task["application_id"]), "submitted",
                        f"用戶在官網完成申請（人工確認）：{merged['external_ref']}",
                        str(attempt["id"]) if attempt else "",
                        "submitted")
                log_audit(db, user_id, "application.submitted", "applications",
                          str(task["application_id"]), {"mode": "manual_confirmed"})
                return {"task": "answered", "application": {"status": "submitted"}}
        except Exception:  # noqa: BLE001
            logger.exception("citics 人工回填處理失敗")
    # 回填內容注入投遞上下文：answer 中與 payload 同名字段直接覆蓋
    db.execute(
        text(
            "UPDATE applications SET status='queued', status_reason='人工補充完成，等待重新投遞', "
            "next_retry_at=NULL, updated_at=now() WHERE id=:aid"
        ),
        {"aid": task["application_id"]},
    )
    db.commit()
    out = run_application(db, str(task["application_id"]))
    return {"task": "answered", "application": out}


# ── 7. 通知 ──────────────────────────────────────────────────────────────


def _notify_result(db: Session, app: dict, success: bool, detail: str) -> None:
    title = f"{'✅ 投遞成功' if success else '❌ 投遞失敗'}：{app['company_name']} · {app['title']}"
    content = (
        f"**{app['company_name']} · {app['title']}**\n"
        f"{'已成功投遞' if success else '投遞失敗，已停止自動重試'}：{detail}\n"
        f"[查看投遞記錄]({app_settings.frontend_url}/applications)"
    )
    notify_with_fallback(db, app["user_id"], "apply_result", title, content,
```

```
                        meta={"application_id": app["id"]})


def _notify_manual(db: Session, app: dict, question: str, task_id: str) -> None:
    title = f"✋ 需要你處理：{app['company_name']} · {app['title']}"
    content = (
        f"**{app['company_name']} · {app['title']}** 的投遞被暫停：\n{question}\n\n"
        f"[點擊處理]({app_settings.frontend_url}/tasks)"
    )
    notify_with_fallback(db, app["user_id"], "manual_task", title, content,
                         meta={"task_id": task_id, "application_id": app["id"]})

"""崗位採集適配器框架（FR-3a2）。

每個招聘來源實現一個 Collector；新增平台只需註冊一個新類，
投遞/匹配/推送流水線完全無感（開閉原則）。
"""
from __future__ import annotations

from dataclasses import dataclass, field
from typing import Protocol


@dataclass
class JobPosting:
    source: str                       # 採集渠道標識（greenhouse/lever/workday/official）
    external_id: str                  # 平台側崗位 ID（與 source 組成去重鍵）
    company_name: str
    title: str
    apply_url: str | None = None
    location: str | None = None
    department: str | None = None
    description_text: str | None = None
    salary_range: dict | None = None
    posted_at: str | None = None      # ISO 字符串
    raw: dict = field(default_factory=dict)


class BaseCollector(Protocol):
    """收集器接口：給定一個目標（careers_url 或公司名），返回崗位列表。"""

    name: str

    def collect(self, target: dict) -> list[JobPosting]:
        """target: {'company_name': str, 'careers_url': str|None, 'platform': str}"""
        ...


_REGISTRY: dict[str, BaseCollector] = {}


def register(collector: BaseCollector) -> BaseCollector:
    _REGISTRY[collector.name] = collector
    return collector


def get_collector(name: str) -> BaseCollector:
    if name not in _REGISTRY:
        raise KeyError(f"未註冊的採集器：{name}（可用：{sorted(_REGISTRY)}）")
    entry = _REGISTRY[name]
    # 註冊的是類時實例化（裝飾器掛在類上）
    if isinstance(entry, type):
        entry = _REGISTRY[name] = entry()
    return entry
```

```
"""官網採集路由：按 URL 特徵自動分發到正確的 ATS 適配器；無法識別時用通用啟發式。

識別規則：
- boards.greenhouse.io / greenhouse.io → Greenhouse
- jobs.lever.co / lever.co             → Lever
- *.myworkdayjobs.com                  → Workday
- 其他 careers_url                     → Generic（Playwright 啟發式，最佳努力）
"""
from __future__ import annotations

import logging

from app.collectors.base import JobPosting, get_collector, register

logger = logging.getLogger(__name__)

_ATS_HINTS = (
    ("greenhouse", ("greenhouse.io",)),
    ("lever", ("lever.co",)),
    ("workday", ("myworkdayjobs.com",)),
    ("gtht", ("hr.gtht.com",)),
    ("citics", ("careers.citics.com",)),
    ("beisen", ("zhiye.com",)),
)

GENERIC_LINK_LIMIT = 10  # 通用採集每站最多抓取詳情數


def detect_platform(careers_url: str) -> str:
    url = careers_url.lower()
    for platform, hints in _ATS_HINTS:
        if any(h in url for h in hints):
            return platform
    return "official"


@register
class OfficialCollector:
    """按 URL 分發；'official' 落到通用啟發式。"""

    name = "official"

    def collect(self, target: dict) -> list[JobPosting]:
        url = target.get("careers_url") or ""
        platform = detect_platform(url)
        if platform != "official":
            return get_collector(platform).collect(target)
        return _generic_collect(target)


def _generic_collect(target: dict) -> list[JobPosting]:
    """通用啟發式：Playwright 打開招聘頁，收集疑似崗位鏈接並抓取標題。

  只做鏈接+標題級採集（JD 詳情留待投遞階段處理），防止請求爆炸。
    """
    try:
        from playwright.sync_api import sync_playwright
    except ImportError as exc:  # pragma: no cover
        raise RuntimeError("通用官網採集需要 playwright：pip install -e '.[automation]'") from exc

    url = target.get("careers_url") or ""
    company = target.get("company_name") or url
    results: list[JobPosting] = []
    job_hint = ("job", "career", "position", "opening", "vacanc", "招聘", "职位")
```

```
with sync_playwright() as pw:
    browser = pw.chromium.launch(headless=True)
    page = browser.new_page()
    try:
        page.goto(url, wait_until="domcontentloaded", timeout=45000)
    except Exception as exc:  # noqa: BLE001
        browser.close()
        raise RuntimeError(f"官網打不開: {url} ({exc})") from exc
    page.wait_for_timeout(2500)
    links = page.eval_on_selector_all(
        "a",
        """els => els.map(e => ({
               href: e.href || '',
               text: (e.innerText || e.getAttribute('aria-label') || '').trim()
             }))
             .filter(l => l.href && l.text && l.text.length < 120)""",
    )
    seen: set[str] = set()
    for link in links:
        low_href = link["href"].lower()
        low_text = link["text"].lower()
        if not any(h in low_href or h in low_text for h in job_hint):
            continue
        key = link["href"].split("?")[0]
        if key in seen:
            continue
        seen.add(key)
        results.append(
            JobPosting(
                source="official",
                external_id=key,
                company_name=company,
                title=link["text"],
                apply_url=link["href"],
            )
        )
        if len(results) >= GENERIC_LINK_LIMIT:
            break
    browser.close()
if not results:
    logger.warning("官網通用採集未發現崗位鏈接: %s", url)
return results
```